

PHẦN II. MẠNG NƠ-RON VÀ HỌC SÂU

CHƯƠNG 10. GIỚI THIỆU VỀ MẠNG NƠ-RON NHÂN TẠO VỚI KERAS

Loài chim đã truyền cảm hứng cho chúng ta bay, cây ngưu bàng truyền cảm hứng cho khóa dán Velcro, và thiên nhiên đã truyền cảm hứng cho vô số phát minh khác. Do đó, việc tìm kiếm nguồn cảm hứng từ kiến trúc của bộ não để xây dựng một cỗ máy thông minh dường như là điều hợp lý. Đây là logic đã châm ngòi cho các mạng nơ-ron nhân tạo (ANNs), các mô hình học máy được lấy cảm hứng từ mạng lưới các nơ-ron sinh học được tìm thấy trong bộ não của chúng ta. Tuy nhiên, mặc dù máy bay được lấy cảm hứng từ loài chim, chúng không cần phải vỗ cánh để bay. Tương tự như vậy, các ANN đã dần trở nên khá khác biệt so với các “họ hàng” sinh học của chúng. Một số nhà nghiên cứu thậm chí còn lập luận rằng chúng ta nên bỏ hẳn phép loại suy sinh học (ví dụ, bằng cách nói “đơn vị” thay vì “nơ-ron”), kéo chúng ta hạn chế sự sáng tạo của mình vào các hệ thống khả thi về mặt sinh học.

ANNs là cốt lõi của học sâu. Chúng linh hoạt, mạnh mẽ và có khả năng mở rộng, làm cho chúng trở nên lý tưởng để giải quyết các tác vụ học máy lớn và phức tạp cao như phân loại hàng tỷ hình ảnh (ví dụ: Google Images), cung cấp dịch vụ nhận dạng giọng nói (ví dụ: Apple Siri), đề xuất các video tốt nhất để hàng trăm triệu người dùng xem mỗi ngày (ví dụ: YouTube), hoặc học cách đánh bại nhà vô địch thế giới trong trò chơi cờ vây (AlphaGo của DeepMind).

Phần đầu tiên của chương này giới thiệu các mạng nơ-ron nhân tạo, bắt đầu với một cái nhìn nhanh về các kiến trúc ANN đầu tiên và dẫn đến các mạng perceptron đa lớp, vốn được sử dụng rộng rãi ngày nay (các kiến trúc khác sẽ được khám phá trong các chương tiếp theo). Trong phần thứ hai, chúng ta sẽ xem xét cách triển khai mạng nơ-ron bằng API Keras của TensorFlow. Đây là một API cấp cao được thiết kế đẹp mắt và đơn giản để xây dựng, huấn luyện, đánh giá và chạy mạng nơ-ron. Nhưng đừng bị đánh lừa bởi sự đơn giản của nó: nó đủ biểu cảm và linh hoạt để cho phép bạn xây dựng nhiều loại kiến trúc mạng nơ-ron. Trên thực tế, nó có thể sẽ đủ cho hầu hết các trường hợp sử dụng của bạn. Và nếu bạn cần thêm sự linh hoạt, bạn luôn có thể viết các thành phần Keras tùy chỉnh bằng API cấp thấp hơn của nó, hoặc thậm chí sử dụng trực tiếp TensorFlow, như bạn sẽ thấy trong Chương 12. Nhưng trước tiên, hãy quay ngược thời gian để xem các mạng nơ-ron nhân tạo ra đời như thế nào!

Từ Nơ-ron Sinh học đến Nơ-ron Nhân tạo

Điều đáng ngạc nhiên là các ANN đã xuất hiện khá lâu: chúng được giới thiệu lần đầu tiên vào năm 1943 bởi nhà thần kinh học Warren McCulloch và nhà toán học Walter Pitts. Trong bài báo mang tính bước ngoặt của họ “A Logical Calculus of Ideas Immanent in Nervous Activity”, McCulloch và Pitts đã trình bày một mô hình tính toán đơn giản về cách các nơ-ron sinh học có thể hoạt động cùng nhau trong não động vật để thực hiện các phép tính phức tạp bằng logic mệnh đề. Đây là kiến trúc mạng nơ-ron nhân tạo đầu tiên. Kể từ đó, nhiều kiến trúc khác đã được phát minh, như bạn sẽ thấy.

Những thành công ban đầu của ANN đã dẫn đến niềm tin rộng rãi rằng chúng ta sẽ sớm trò chuyện với những cỗ máy thực sự thông minh. Khi vào những năm 1960, rõ ràng là lời hứa này sẽ không được thực hiện (ít nhất là trong một thời gian khá dài), các khoản tài trợ đã chuyển hướng, và ANN bước vào một mùa đông dài. Đầu những năm 1980, các kiến trúc mới đã được phát minh và các kỹ thuật huấn luyện tốt hơn đã được phát triển, châm ngòi cho sự hồi sinh của mối quan tâm đến thuyết liên kết (connectionism) – nghiên cứu về mạng nơ-ron. Nhưng tiến độ chậm, và đến những năm 1990, các kỹ thuật học máy mạnh mẽ khác đã được phát minh, chẳng hạn như máy vector hỗ trợ (xem Chương 5). Các kỹ thuật này dường như mang lại kết quả tốt hơn và nền tảng lý thuyết vững chắc hơn so với ANN, vì vậy một lần nữa, nghiên cứu về mạng nơ-ron lại bị tạm dừng.

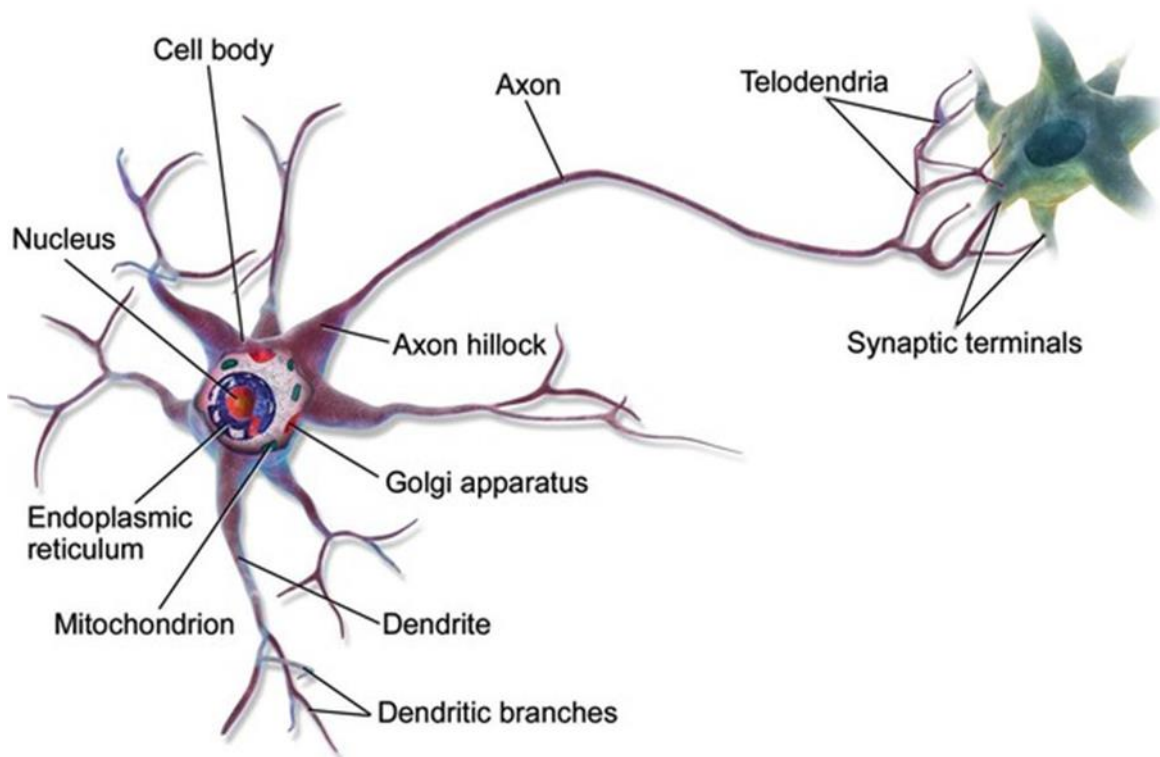
Hiện tại chúng ta đang chứng kiến một làn sóng quan tâm khác đến ANN. Liệu làn sóng này có biến mất như những làn sóng trước không? Chà, đây là một vài lý do tốt để tin rằng làn sóng này sẽ khác và mối quan tâm mới đến ANN sẽ có tác động sâu sắc hơn nhiều đến cuộc sống của chúng ta:

- Hiện có một lượng lớn dữ liệu có sẵn để huấn luyện mạng nơ-ron, và ANN thường vượt trội hơn các kỹ thuật học máy khác trong các vấn đề rất lớn và phức tạp.
- Sự gia tăng đáng kể về sức mạnh tính toán kể từ những năm 1990 hiện cho phép huấn luyện các mạng nơ-ron lớn trong một khoảng thời gian hợp lý. Điều này một phần là do định luật Moore (số lượng linh kiện trong mạch tích hợp đã tăng gấp đôi khoảng 2 năm một lần trong 50 năm qua), nhưng cũng nhờ vào ngành công nghiệp trò chơi, đã thúc đẩy sản xuất hàng triệu thẻ GPU mạnh mẽ. Hơn nữa, các nền tảng đám mây đã giúp mọi người có thể tiếp cận sức mạnh này.
- Các thuật toán huấn luyện đã được cải thiện. Công bằng mà nói, chúng chỉ hơi khác so với những thuật toán được sử dụng vào những năm 1990, nhưng những thay đổi tương đối nhỏ này đã có tác động tích cực rất lớn.
- Một số hạn chế lý thuyết của ANN đã trở nên vô hại trong thực tế. Ví dụ, nhiều người nghĩ rằng các thuật toán huấn luyện ANN sẽ thất bại vì chúng có khả năng bị mắc kẹt trong các cực tiểu cục bộ, nhưng hóa ra đây không phải là vấn đề lớn trong thực tế, đặc biệt đối với các mạng nơ-ron lớn hơn: các cực tiểu cục bộ thường hoạt động tốt gần bằng cực đại toàn cục.
- ANNs dường như đã đi vào một vòng luẩn quẩn tích cực của việc tài trợ và tiến bộ. Các sản phẩm tuyệt vời dựa trên ANN thường xuyên xuất hiện trên các bản tin, thu hút ngày càng nhiều sự chú ý và tài trợ cho chúng, dẫn đến ngày càng nhiều tiến bộ và thậm chí nhiều sản phẩm tuyệt vời hơn.

Nơ-ron Sinh học

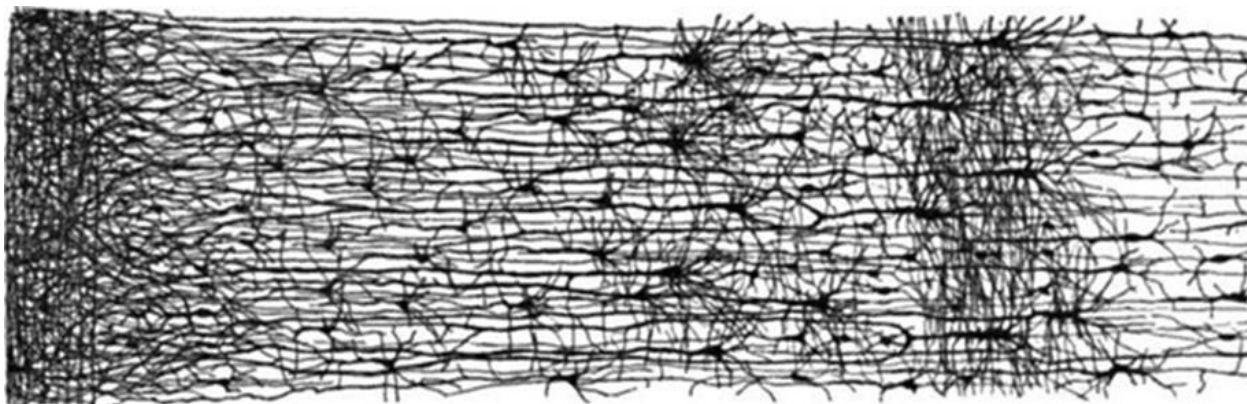
Trước khi chúng ta thảo luận về các nơ-ron nhân tạo, hãy xem nhanh một nơ-ron sinh học (được trình bày trong Hình 10-1). Nó là một tế bào có hình dạng bất thường chủ yếu được tìm thấy trong não động vật. Nó bao gồm một thân tế bào chứa nhân và hầu hết các thành phần phức tạp của tế bào, nhiều phần mở rộng phân nhánh được gọi là đuôi gai (dendrites), cộng với một phần mở rộng rất dài được gọi là sợi trục (axon). Chiều dài của sợi trục có thể chỉ dài hơn vài lần so với thân tế bào, hoặc lên đến hàng chục nghìn lần. Gần

cuối sợi trục, nó phân nhánh thành nhiều nhánh nhỏ hơn được gọi là telodendria, và ở đầu các nhánh này là các cấu trúc nhỏ xíu được gọi là đầu cuối khớp thần kinh (synaptic terminals) (hoặc đơn giản là khớp thần kinh), được kết nối với đuôi gai hoặc thân tế bào của các nơ-ron khác. Các nơ-ron sinh học tạo ra các xung điện ngắn gọi là điện thế hoạt động (APs, hoặc đơn giản là tín hiệu), di chuyển dọc theo các sợi trục và khiến các khớp thần kinh giải phóng các tín hiệu hóa học gọi là chất dẫn truyền thần kinh. Khi một nơ-ron nhận đủ lượng chất dẫn truyền thần kinh này trong vòng vài mili giây, nó sẽ phát ra các xung điện của riêng mình (thực tế, điều này phụ thuộc vào chất dẫn truyền thần kinh, vì một số trong số chúng ức chế nơ-ron phát xung).



Hình 10-1. Một nơ-ron sinh học

Do đó, các nơ-ron sinh học riêng lẻ dường như hoạt động theo một cách đơn giản, nhưng chúng được tổ chức trong một mạng lưới rộng lớn gồm hàng tỷ nơ-ron, với mỗi nơ-ron thường được kết nối với hàng nghìn nơ-ron khác. Các phép tính cực kỳ phức tạp có thể được thực hiện bởi một mạng lưới các nơ-ron khá đơn giản, giống như một tổ kiến phức tạp có thể xuất hiện từ những nỗ lực kết hợp của những con kiến đơn giản. Kiến trúc của mạng nơ-ron sinh học (BNN) là chủ đề nghiên cứu tích cực, nhưng một số phần của bộ não đã được lập bản đồ. Những nỗ lực này cho thấy rằng các nơ-ron thường được tổ chức thành các lớp liên tiếp, đặc biệt là ở vỏ não (lớp ngoài của não), như được trình bày trong Hình 10-2.

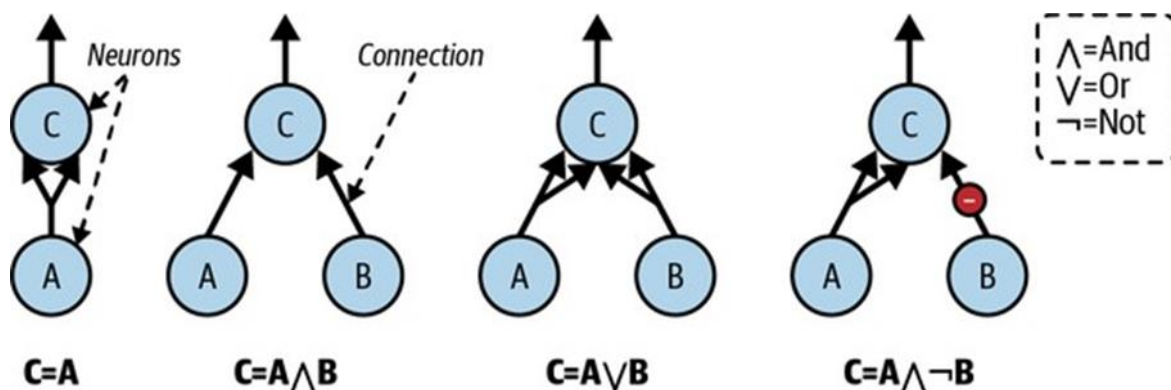


Hình 10-2. Nhiều lớp trong mạng nơ-ron sinh học (vỏ não người)

Các phép tính logic với nơ-ron

McCulloch và Pitts đã đề xuất một mô hình rất đơn giản của nơ-ron sinh học, sau này được gọi là nơ-ron nhân tạo: nó có một hoặc nhiều đầu vào nhị phân (bật/tắt) và một đầu ra nhị phân. Nơ-ron nhân tạo kích hoạt đầu ra của nó khi nhiều hơn một số lượng nhất định các đầu vào của nó đang hoạt động. Trong bài báo của họ, McCulloch và Pitts đã chỉ ra rằng ngay cả với một mô hình đơn giản như vậy, có thể xây dựng một mạng lưới các nơ-ron nhân tạo có thể tính toán bất kỳ mệnh đề logic nào bạn muốn.

Để xem mạng lưới như vậy hoạt động như thế nào, hãy xây dựng một vài ANN thực hiện các phép tính logic khác nhau (xem Hình 10-3), giả sử rằng một nơ-ron được kích hoạt khi ít nhất hai trong số các kết nối đầu vào của nó đang hoạt động.



Hình 10-3. ANN thực hiện các phép tính logic đơn giản

Hãy xem các mạng này làm gì:

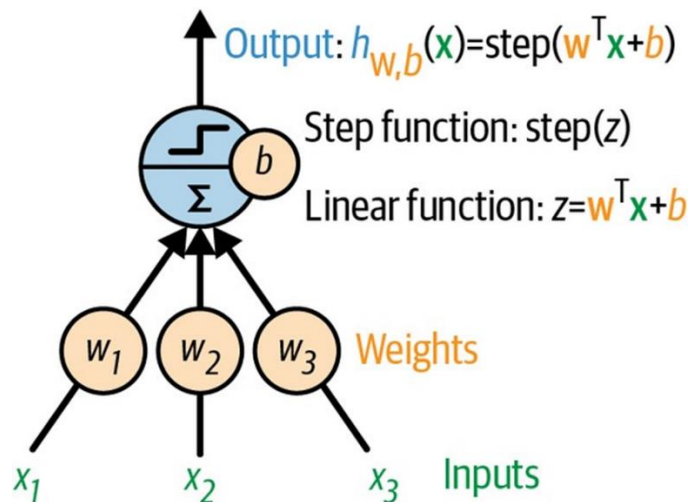
- Mạng đầu tiên ở bên trái là hàm đồng nhất (identity function): nếu nơ-ron A được kích hoạt, thì nơ-ron C cũng được kích hoạt (vì nó nhận hai tín hiệu đầu vào từ nơ-ron A); nhưng nếu nơ-ron A tắt, thì nơ-ron C cũng tắt.

- Mạng thứ hai thực hiện phép toán logic VÀ (AND): nơ-ron C chỉ được kích hoạt khi cả nơ-ron A và B đều được kích hoạt (một tín hiệu đầu vào duy nhất không đủ để kích hoạt nơ-ron C).
- Mạng thứ ba thực hiện phép toán logic HOẶC (OR): nơ-ron C được kích hoạt nếu nơ-ron A hoặc nơ-ron B được kích hoạt (hoặc cả hai).
- Cuối cùng, nếu chúng ta giả sử rằng một kết nối đầu vào có thể ức chế hoạt động của nơ-ron (điều này đúng với nơ-ron sinh học), thì mạng thứ tư tính toán một mệnh đề logic phức tạp hơn một chút: nơ-ron C chỉ được kích hoạt nếu nơ-ron A hoạt động và nơ-ron B tắt. Nếu nơ-ron A hoạt động mọi lúc, thì bạn nhận được một phép toán logic KHÔNG (NOT): nơ-ron C hoạt động khi nơ-ron B tắt và ngược lại.

Bạn có thể tưởng tượng cách các mạng này có thể được kết hợp để tính toán các biểu thức logic phức tạp (xem các bài tập ở cuối chương để biết ví dụ).

Perceptron

Perceptron là một trong những kiến trúc mạng nơ-ron nhân tạo (ANN) đơn giản nhất, được Frank Rosenblatt phát minh vào năm 1957. Nó dựa trên một nơ-ron nhân tạo hơi khác một chút (xem Hình 10-4) được gọi là đơn vị logic ngưỡng (TLU), hoặc đôi khi là đơn vị ngưỡng tuyến tính (LTU). Các đầu vào và đầu ra là các số (thay vì các giá trị bật/tắt nhị phân), và mỗi kết nối đầu vào được liên kết với một trọng số. TLU trước tiên tính toán một hàm tuyến tính của các đầu vào của nó: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w}^T\mathbf{x} + b$. Sau đó, nó áp dụng một hàm bước cho kết quả: $h_{\mathbf{w},b}(\mathbf{x}) = \text{step}(z)$. Vì vậy, nó gần giống như hồi quy logistic, ngoại trừ việc nó sử dụng hàm bước thay vì hàm logistic (Chương 4). Giống như trong hồi quy logistic, các tham số của mô hình là trọng số đầu vào w và số hạng sai lệch b .



Hình 10-4. TLU: một nơ-ron nhân tạo tính tổng có trọng số của các đầu vào $\mathbf{w}^T\mathbf{x}$, cộng với số hạng sai lệch b , sau đó áp dụng hàm bước

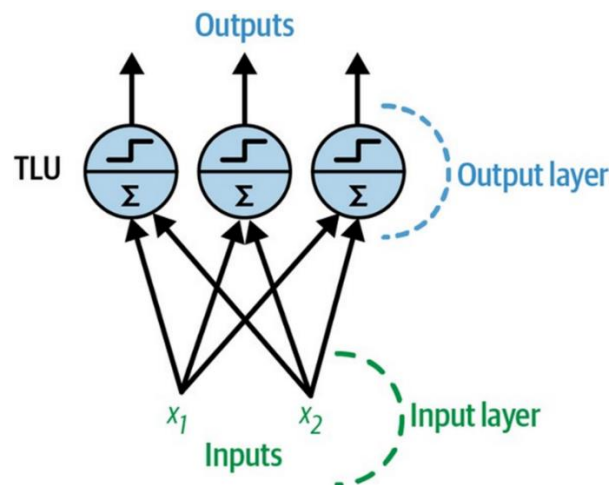
Hàm bước phổ biến nhất được sử dụng trong perceptron là hàm bước Heaviside (xem Công thức 10-1). Đôi khi hàm dấu được sử dụng thay thế.

Công thức 10-1. Các hàm bước phổ biến được sử dụng trong perceptron (giả sử ngưỡng = 0)

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \quad \text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

Một TLU đơn có thể được sử dụng cho phân loại nhị phân tuyến tính đơn giản. Nó tính toán một hàm tuyến tính của các đầu vào của nó, và nếu kết quả vượt quá ngưỡng, nó sẽ xuất ra lớp tích cực. Ngược lại, nó xuất ra lớp tiêu cực. Điều này có thể gợi cho bạn nhớ đến hồi quy logistic (Chương 4) hoặc phân loại SVM tuyến tính (Chương 5). Bạn có thể, ví dụ, sử dụng một TLU đơn để phân loại hoa diên vĩ dựa trên chiều dài và chiều rộng cánh hoa. Huấn luyện một TLU như vậy sẽ yêu cầu tìm các giá trị đúng cho w_1, w_2 , và b (thuật toán huấn luyện sẽ được thảo luận ngay sau đây).

Một perceptron được cấu tạo từ một hoặc nhiều TLU được tổ chức trong một lớp duy nhất, trong đó mỗi TLU được kết nối với mọi đầu vào. Một lớp như vậy được gọi là lớp kết nối đầy đủ, hoặc lớp dày đặc. Các đầu vào tạo thành lớp đầu vào. Và vì lớp TLU tạo ra các đầu ra cuối cùng, nó được gọi là lớp đầu ra. Ví dụ, một perceptron với hai đầu vào và ba đầu ra được biểu diễn trong Hình 10-5.



Hình 10-5. Kiến trúc của một perceptron với hai đầu vào và ba nơ-ron đầu ra

Perceptron này có thể phân loại các trường hợp đồng thời thành ba lớp nhị phân khác nhau, điều này làm cho nó trở thành một bộ phân loại đa nhãn. Nó cũng có thể được sử dụng cho phân loại đa lớp. Nhờ sự kỳ diệu của đại số tuyến tính, Công thức 10-2 có thể được sử dụng để tính toán hiệu quả các đầu ra của một lớp nơ-ron nhân tạo cho nhiều trường hợp cùng một lúc.

Công thức 10-2. Tính toán đầu ra của một lớp kết nối đầy đủ

$$h_{W,b}(X) = \text{phi}(XW + b)$$

Trong phương trình này:

- Như thường lệ, X biểu thị ma trận các đặc trưng đầu vào. Nó có một hàng cho mỗi trường hợp và một cột cho mỗi đặc trưng.
- Ma trận trọng số W chứa tất cả các trọng số kết nối. Nó có một hàng cho mỗi đầu vào và một cột cho mỗi nơ-ron.
- Vector độ lệch b chứa tất cả các số hạng độ lệch: một cho mỗi nơ-ron.
- Hàm ϕ được gọi là hàm kích hoạt: khi các nơ-ron nhân tạo là TLU, nó là một hàm bước (chúng ta sẽ thảo luận về các hàm kích hoạt khác sau).

Vậy, một perceptron được huấn luyện như thế nào? Thuật toán huấn luyện perceptron được Rosenblatt đề xuất phần lớn được lấy cảm hứng từ quy tắc Hebb. Trong cuốn sách năm 1949 của ông *The Organization of Behavior* (Wiley), Donald Hebb gợi ý rằng khi một nơ-ron sinh học kích hoạt một nơ-ron khác thường xuyên, kết nối giữa hai nơ-ron này sẽ mạnh hơn. Siegrid Löwel sau đó đã tóm tắt ý tưởng của Hebb bằng cụm từ hấp dẫn, “Các tế bào kích hoạt cùng nhau sẽ kết nối cùng nhau” (Cells that fire together, wire together); nghĩa là, trọng số kết nối giữa hai nơ-ron có xu hướng tăng lên khi chúng kích hoạt đồng thời. Quy tắc này sau đó được gọi là quy tắc Hebb (hoặc học Hebbian).

Perceptron được huấn luyện bằng cách sử dụng một biến thể của quy tắc này có tính đến lỗi mà mạng mắc phải khi đưa ra dự đoán; quy tắc học perceptron củng cố các kết nối giúp giảm lỗi. Cụ thể hơn, perceptron được đưa một trường hợp huấn luyện một lúc, và đối với mỗi trường hợp, nó đưa ra dự đoán của mình. Đối với mỗi nơ-ron đầu ra tạo ra dự đoán sai, nó sẽ củng cố trọng số kết nối từ các đầu vào lẽ ra đã góp phần vào dự đoán chính xác. Quy tắc được trình bày trong Công thức 10-3.

Công thức 10-3. Quy tắc học perceptron (cập nhật trọng số)

$$w_{i,j}^{(next\ step)} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

Trong phương trình này:

- $w_{i,j}$ là trọng số kết nối giữa đầu vào thứ i và nơ-ron thứ j .
- x_i là giá trị đầu vào thứ i của trường hợp huấn luyện hiện tại.
- $\hat{y}_j$ là đầu ra của nơ-ron đầu ra thứ j cho trường hợp huấn luyện hiện tại.
- y_j là đầu ra mục tiêu của nơ-ron đầu ra thứ j cho trường hợp huấn luyện hiện tại.
- η là tốc độ học (xem Chương 4).

Ranh giới quyết định của mỗi nơ-ron đầu ra là tuyến tính, do đó perceptron không có khả năng học các mẫu phức tạp (giống như bộ phân loại hồi quy logistic). Tuy nhiên, nếu các trường hợp huấn luyện có thể phân tách tuyến tính, Rosenblatt đã chứng minh rằng thuật toán này sẽ hội tụ đến một giải pháp. Điều này được gọi là định lý hội tụ perceptron.

Scikit-Learn cung cấp lớp Perceptron có thể được sử dụng gần như theo cách bạn mong đợi - ví dụ, trên tập dữ liệu iris (được giới thiệu trong Chương 4):

```

import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 0) # Iris setosa

per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)

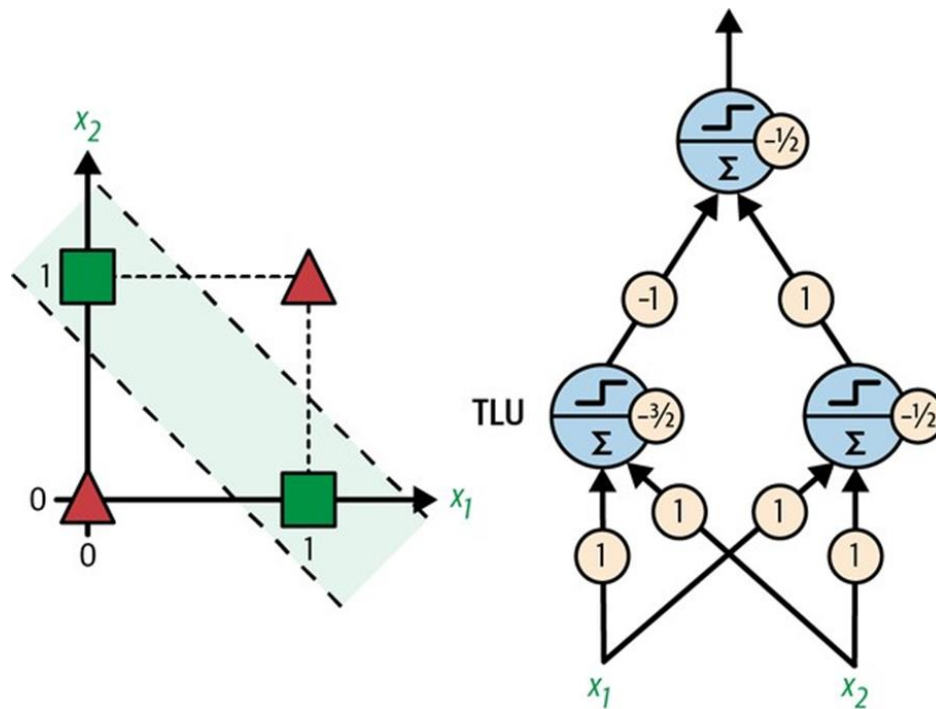
X_new = [[2, 0.5], [3, 1]]
y_pred = per_clf.predict(X_new) # predicts True and False for these 2 flowers

```

Bạn có thể nhận thấy rằng thuật toán học perceptron rất giống với thuật toán giảm độ dốc ngẫu nhiên (được giới thiệu trong Chương 4). Trên thực tế, lớp Perceptron của Scikit-Learn tương đương với việc sử dụng `SGDClassifier` với các siêu tham số sau: `loss="perceptron"`, `learning_rate="constant"`, `eta0=1` (tốc độ học), và `penalty=None` (không chính quy hóa).

Trong chuyên khảo *Perceptrons* năm 1969 của họ, Marvin Minsky và Seymour Papert đã nêu bật một số điểm yếu nghiêm trọng của perceptron - đặc biệt, thực tế là chúng không có khả năng giải quyết một số vấn đề tầm thường (ví dụ: vấn đề phân loại HOẶC LOẠI TRỪ (XOR); xem phía bên trái của Hình 10-6). Điều này đúng với bất kỳ mô hình phân loại tuyến tính nào khác (chẳng hạn như bộ phân loại hồi quy logistic), nhưng các nhà nghiên cứu đã kỳ vọng nhiều hơn từ perceptron, và một số người đã thất vọng đến mức họ từ bỏ hoàn toàn mạng nơ-ron để chuyển sang các vấn đề cấp cao hơn như logic, giải quyết vấn đề và tìm kiếm. Việc thiếu các ứng dụng thực tế cũng không giúp ích gì.

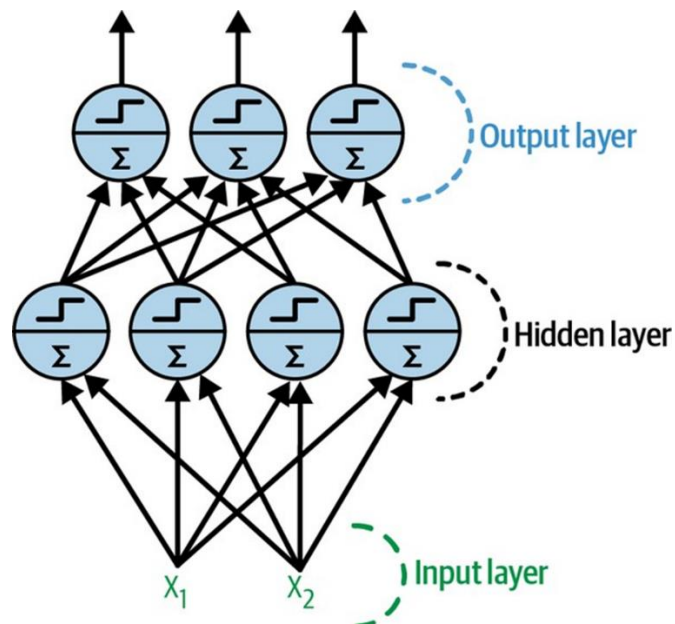
Hóa ra một số hạn chế của perceptron có thể được loại bỏ bằng cách xếp chồng nhiều perceptron. Mạng nơ-ron nhân tạo thu được được gọi là mạng perceptron đa lớp (MLP). Một MLP có thể giải quyết vấn đề XOR, như bạn có thể xác minh bằng cách tính toán đầu ra của MLP được biểu diễn ở phía bên phải của Hình 10-6: với đầu vào (0, 0) hoặc (1, 1), mạng xuất ra 0, và với đầu vào (0, 1) hoặc (1, 0) nó xuất ra 1. Hãy thử xác minh rằng mạng này thực sự giải quyết vấn đề XOR!



Hình 10-6. Vấn đề phân loại XOR và một MLP giải quyết nó

Mạng perceptron đa lớp và Backpropagation

Một MLP được cấu tạo từ một lớp đầu vào, một hoặc nhiều lớp TLU được gọi là lớp ẩn, và một lớp TLU cuối cùng được gọi là lớp đầu ra (xem Hình 10-7). Các lớp gần lớp đầu vào thường được gọi là lớp dưới, và các lớp gần đầu ra thường được gọi là lớp trên.



Hình 10-7. Kiến trúc của một mạng perceptron đa lớp với hai đầu vào, một lớp ẩn gồm bốn nơ-ron và ba nơ-ron đầu ra

Khi một ANN chứa một chồng sâu các lớp ẩn, nó được gọi là mạng nơ-ron sâu (DNN). Lĩnh vực học sâu nghiên cứu DNN, và tổng quát hơn là quan tâm đến các mô hình chứa các chồng tính toán sâu. Mặc dù vậy, nhiều người nói về học sâu bất cứ khi nào có liên quan đến mạng nơ-ron (ngay cả những mạng nông).

Trong nhiều năm, các nhà nghiên cứu đã cố gắng tìm cách huấn luyện MLP nhưng không thành công. Đầu những năm 1960, một số nhà nghiên cứu đã thảo luận về khả năng sử dụng gradient descent để huấn luyện mạng nơ-ron, nhưng như chúng ta đã thấy trong Chương 4, điều này đòi hỏi phải tính toán gradient của lỗi mô hình đối với các tham số mô hình; vào thời điểm đó, không rõ làm thế nào để thực hiện việc này một cách hiệu quả với một mô hình phức tạp như vậy chứa quá nhiều tham số, đặc biệt với các máy tính mà họ có khi đó.

Sau đó, vào năm 1970, một nhà nghiên cứu tên là Seppo Linnainmaa đã giới thiệu trong luận văn thạc sĩ của mình một kỹ thuật để tính toán tất cả các gradient một cách tự động và hiệu quả. Thuật toán này hiện được gọi là tự động đạo hàm ngược (reverse-mode automatic differentiation) (hoặc viết tắt là reverse-mode autodiff). Chỉ trong hai lượt qua mạng (một lượt tiến, một lượt lùi), nó có thể tính toán gradient của lỗi mạng nơ-ron đối với từng tham số mô hình. Nói cách khác, nó có thể tìm ra cách điều chỉnh từng trọng số kết nối và từng độ lệch để giảm lỗi của mạng nơ-ron. Các gradient này sau đó có thể được sử dụng để thực hiện một bước giảm độ dốc. Nếu bạn lặp lại quá trình tính toán gradient tự động và thực hiện một bước giảm độ dốc này, lỗi của mạng nơ-ron sẽ giảm dần cho đến khi cuối cùng đạt đến một cực tiểu. Sự kết hợp giữa tự động đạo hàm ngược và giảm độ dốc này hiện được gọi là backpropagation (hoặc viết tắt là backprop).

Backpropagation thực sự có thể được áp dụng cho tất cả các loại biểu đồ tính toán, không chỉ mạng nơ-ron: thực tế, luận văn thạc sĩ của Linnainmaa không phải về mạng nơ-ron, nó mang tính tổng quát hơn. Phải mất thêm vài năm nữa trước khi backprop bắt đầu được sử dụng để huấn luyện mạng nơ-ron, nhưng nó vẫn chưa phổ biến. Sau đó, vào năm 1985, David Rumelhart, Geoffrey Hinton và Ronald Williams đã công bố một bài báo đột phá phân tích cách backpropagation cho phép mạng nơ-ron học các biểu diễn nội bộ hữu ích. Kết quả của họ ảnh hưởng đến mức backpropagation nhanh chóng được phổ biến trong lĩnh vực này. Ngày nay, nó là kỹ thuật huấn luyện phổ biến nhất cho mạng nơ-ron.

Hãy cùng xem lại cách backpropagation hoạt động chi tiết hơn một chút:

- Nó xử lý từng mini-batch một (ví dụ, mỗi mini-batch chứa 32 trường hợp), và nó đi qua toàn bộ tập huấn luyện nhiều lần. Mỗi lượt được gọi là một epoch.
- Mỗi mini-batch đi vào mạng thông qua lớp đầu vào. Thuật toán sau đó tính toán đầu ra của tất cả các nơ-ron trong lớp ẩn đầu tiên, cho mọi trường hợp trong mini-batch. Kết quả được chuyển đến lớp tiếp theo, đầu ra của nó được tính toán và chuyển đến lớp tiếp theo, cứ thế cho đến khi chúng ta nhận được đầu ra của lớp cuối cùng, lớp đầu ra. Đây là lượt truyền tiến (forward pass): nó giống hệt như việc đưa ra dự đoán, ngoại trừ tất cả các kết quả trung gian được bảo toàn vì chúng cần thiết cho lượt truyền ngược.

- Tiếp theo, thuật toán đo lỗi đầu ra của mạng (nghĩa là, nó sử dụng một hàm mất mát so sánh đầu ra mong muốn và đầu ra thực tế của mạng, và trả về một số thước đo lỗi).
- Sau đó, nó tính toán mức độ mỗi độ lệch đầu ra và mỗi kết nối đến lớp đầu ra đã đóng góp vào lỗi. Điều này được thực hiện bằng cách phân tích áp dụng quy tắc chuỗi (có lẽ là quy tắc cơ bản nhất trong giải tích), điều này làm cho bước này nhanh chóng và chính xác.
- Thuật toán sau đó đo lường mức độ các đóng góp lỗi này đến từ mỗi kết nối trong lớp bên dưới, một lần nữa sử dụng quy tắc chuỗi, hoạt động ngược lại cho đến khi nó đạt đến lớp đầu vào. Như đã giải thích trước đó, lượt truyền ngược này đo lường hiệu quả gradient lỗi trên tất cả các trọng số kết nối và độ lệch trong mạng bằng cách truyền ngược gradient lỗi qua mạng (do đó có tên của thuật toán).
- Cuối cùng, thuật toán thực hiện một bước giảm độ dốc để điều chỉnh tất cả các trọng số kết nối trong mạng, sử dụng các gradient lỗi vừa được tính toán.

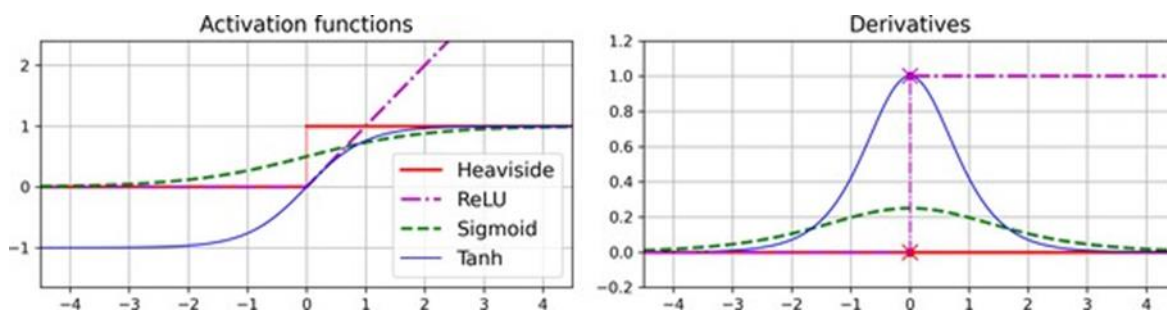
Tóm lại, backpropagation đưa ra dự đoán cho một mini-batch (lượt truyền tiến), đo lỗi, sau đó đi qua từng lớp theo chiều ngược lại để đo đóng góp lỗi từ mỗi tham số (lượt truyền ngược), và cuối cùng điều chỉnh trọng số kết nối và độ lệch để giảm lỗi (bước giảm độ dốc).

Để backprop hoạt động đúng cách, Rumelhart và các đồng nghiệp của ông đã thực hiện một thay đổi quan trọng đối với kiến trúc của MLP: họ thay thế hàm bước bằng hàm logistic, $\sigma(z) = 1/(1 + \exp(-z))$, còn được gọi là hàm sigmoid. Điều này rất cần thiết vì hàm bước chỉ chứa các đoạn phẳng, do đó không có gradient để làm việc (giảm độ dốc không thể di chuyển trên bề mặt phẳng), trong khi hàm sigmoid có đạo hàm khác 0 được xác định rõ ở khắp mọi nơi, cho phép giảm độ dốc đạt được một số tiến bộ ở mỗi bước. Trên thực tế, thuật toán backpropagation hoạt động tốt với nhiều hàm kích hoạt khác, không chỉ hàm sigmoid. Dưới đây là hai lựa chọn phổ biến khác:

- **Hàm hyperbolic tangent:** $\tanh(z) = 2\sigma(2z) - 1$ Giống như hàm sigmoid, hàm kích hoạt này có hình chữ S, liên tục và khả vi, nhưng giá trị đầu ra của nó nằm trong khoảng từ -1 đến 1 (thay vì 0 đến 1 trong trường hợp hàm sigmoid). Khoảng giá trị đó có xu hướng làm cho đầu ra của mỗi lớp ít nhiều tập trung quanh 0 khi bắt đầu huấn luyện, điều này thường giúp tăng tốc độ hội tụ.
- **Hàm rectified linear unit (ReLU):** $\text{ReLU}(z) = \max(0, z)$ Hàm ReLU liên tục nhưng không khả vi tại $z = 0$ (độ dốc thay đổi đột ngột, điều này có thể khiến gradient descent bị dao động), và đạo hàm của nó bằng 0 với $z < 0$. Tuy nhiên, trong thực tế, nó hoạt động rất tốt và có ưu điểm là tính toán nhanh, vì vậy nó đã trở thành mặc định. Quan trọng là, việc nó không có giá trị đầu ra tối đa giúp giảm một số vấn đề trong quá trình giảm độ dốc (chúng ta sẽ quay lại vấn đề này trong Chương 11).

Các hàm kích hoạt phổ biến này và đạo hàm của chúng được biểu diễn trong Hình 10-8. Nhưng khoan! Tại sao chúng ta cần các hàm kích hoạt ngay từ đầu? Chà, nếu bạn nghĩ nhiều phép biến đổi tuyến tính, tất cả những gì bạn nhận được là một phép biến đổi tuyến tính. Ví

dụ, nếu $f(x) = 2x + 3$ và $g(x) = 5x - 1$, thì việc nối hai hàm tuyến tính này sẽ cho bạn một hàm tuyến tính khác: $f(g(x)) = 2(5x - 1) + 3 = 10x + 1$. Vì vậy, nếu bạn không có một số phi tuyến tính giữa các lớp, thì ngay cả một chồng sâu các lớp cũng tương đương với một lớp duy nhất, và bạn không thể giải quyết các vấn đề rất phức tạp với điều đó. Ngược lại, một DNN đủ lớn với các kích hoạt phi tuyến tính về mặt lý thuyết có thể xấp xỉ bất kỳ hàm liên tục nào.



Hình 10-8. Các hàm kích hoạt (trái) và đạo hàm của chúng (phải)

OK! Bạn đã biết mạng nơ-ron từ đâu mà có, kiến trúc của chúng là gì và cách tính toán đầu ra của chúng. Bạn cũng đã học về thuật toán backpropagation. Nhưng chính xác thì bạn có thể làm gì với mạng nơ-ron?

MLP hồi quy

Đầu tiên, MLP có thể được sử dụng cho các tác vụ hồi quy. Nếu bạn muốn dự đoán một giá trị duy nhất (ví dụ: giá của một căn nhà, với nhiều đặc trưng của nó), thì bạn chỉ cần một nơ-ron đầu ra duy nhất: đầu ra của nó là giá trị dự đoán. Đối với hồi quy đa biến (nghĩa là, để dự đoán nhiều giá trị cùng một lúc), bạn cần một nơ-ron đầu ra cho mỗi chiều đầu ra. Ví dụ, để định vị trung tâm của một đối tượng trong hình ảnh, bạn cần dự đoán tọa độ 2D, vì vậy bạn cần hai nơ-ron đầu ra. Nếu bạn cũng muốn đặt một hộp giới hạn xung quanh đối tượng, thì bạn cần thêm hai số nữa: chiều rộng và chiều cao của đối tượng. Vì vậy, bạn sẽ có bốn nơ-ron đầu ra.

Scikit-Learn bao gồm một lớp `MLPRegressor`, vì vậy hãy sử dụng nó để xây dựng một MLP với ba lớp ẩn gồm 50 nơ-ron mỗi lớp, và huấn luyện nó trên tập dữ liệu California housing. Để đơn giản, chúng ta sẽ sử dụng hàm `fetch_california_housing()` của Scikit-Learn để tải dữ liệu. Tập dữ liệu này đơn giản hơn tập dữ liệu chúng ta đã sử dụng trong Chương 2, vì nó chỉ chứa các đặc trưng số (không có đặc trưng `ocean_proximity`), và không có giá trị bị thiếu. Đoạn mã sau bắt đầu bằng cách lấy và chia tập dữ liệu, sau đó nó tạo một pipeline để chuẩn hóa các đặc trưng đầu vào trước khi gửi chúng đến `MLPRegressor`. Điều này rất quan trọng đối với mạng nơ-ron vì chúng được huấn luyện bằng cách sử dụng giảm độ dốc, và như chúng ta đã thấy trong Chương 4, giảm độ dốc không hội tụ tốt khi các đặc trưng có các thang đo rất khác nhau. Cuối cùng, mã này huấn luyện mô hình và đánh giá lỗi xác thực của nó. Mô hình sử dụng hàm kích hoạt ReLU trong các lớp ẩn, và nó sử dụng một biến thể của giảm độ dốc gọi là Adam (xem Chương 11) để giảm thiểu lỗi bình phương trung bình, với một chút chính quy hóa l_2 (mà bạn có thể kiểm soát thông qua siêu tham số `alpha`):

```

from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(housing.data,
housing.target, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(X_train_full,
y_train_full, random_state=42)

mlp_reg = MLPRegressor(hidden_layer_sizes=[50, 50, 50], random_state=42)
pipeline = make_pipeline(StandardScaler(), mlp_reg)
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_valid)

rmse = mean_squared_error(y_valid, y_pred, squared=False) # about 0.505

```

Chúng ta nhận được RMSE xác thực khoảng 0.505, có thể so sánh với những gì bạn sẽ nhận được với một bộ phân loại rừng ngẫu nhiên. Không tệ cho lần thử đầu tiên!

Lưu ý rằng MLP này không sử dụng bất kỳ hàm kích hoạt nào cho lớp đầu ra, vì vậy nó được tự do xuất ra bất kỳ giá trị nào nó muốn. Điều này thường ổn, nhưng nếu bạn muốn đảm bảo rằng đầu ra sẽ luôn dương, thì bạn nên sử dụng hàm kích hoạt ReLU trong lớp đầu ra, hoặc hàm kích hoạt softplus, là một biến thể mượt mà của ReLU: $\text{softplus}(z) = \log(1 + \exp(z))$. Softplus gần bằng 0 khi z âm và gần bằng z khi z dương. Cuối cùng, nếu bạn muốn đảm bảo rằng các dự đoán sẽ luôn nằm trong một phạm vi giá trị nhất định, thì bạn nên sử dụng hàm sigmoid hoặc hàm hyperbolic tangent, và điều chỉnh các mục tiêu về phạm vi thích hợp: 0 đến 1 cho sigmoid và -1 đến 1 cho tanh. Đáng buồn thay, lớp MLPRegressor không hỗ trợ các hàm kích hoạt trong lớp đầu ra.

Lớp MLPRegressor sử dụng lỗi bình phương trung bình, đây thường là điều bạn muốn cho hồi quy, nhưng nếu bạn có nhiều giá trị ngoại lai trong tập huấn luyện, bạn có thể muốn sử dụng lỗi tuyệt đối trung bình thay thế. Ngoài ra, bạn có thể muốn sử dụng Huber loss, là sự kết hợp của cả hai. Nó là hàm bậc hai khi lỗi nhỏ hơn một ngưỡng δ (thường là 1) nhưng là hàm tuyến tính khi lỗi lớn hơn δ . Phần tuyến tính làm cho nó ít nhạy cảm với các giá trị ngoại lai hơn lỗi bình phương trung bình, và phần bậc hai cho phép nó hội tụ nhanh hơn và chính xác hơn lỗi tuyệt đối trung bình. Tuy nhiên, MLPRegressor chỉ hỗ trợ MSE.

Bảng 10-1 tóm tắt kiến trúc điển hình của một MLP hồi quy.

Bảng 10-1. Kiến trúc MLP hồi quy điển hình

Siêu tham số	Giá trị điển hình
# lớp ẩn	Tùy thuộc vào vấn đề, nhưng thường từ 1 đến 5

Siêu tham số	Giá trị điển hình
# nơ-ron đầu ra	1 cho mỗi chiều dự đoán
Kích hoạt đầu ra	Không, hoặc ReLU/softplus (nếu đầu ra dương) hoặc sigmoid/tanh (nếu đầu ra có giới hạn)

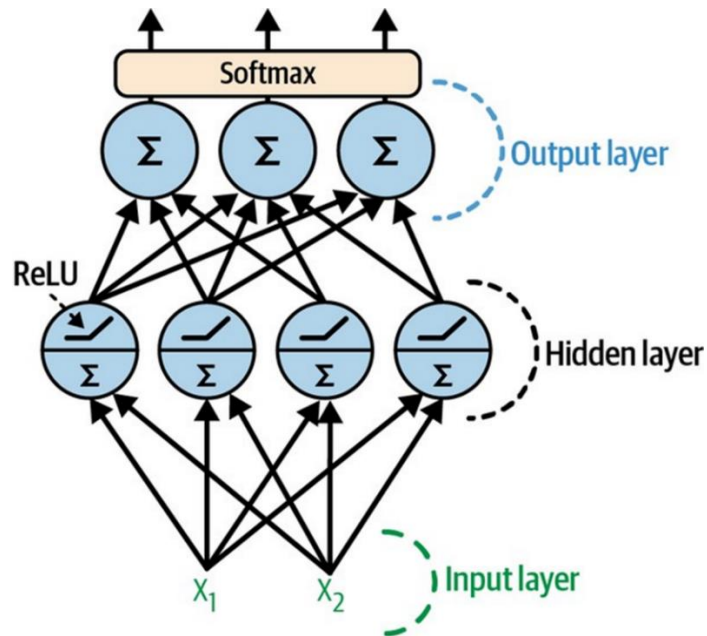
MLP phân loại

MLP cũng có thể được sử dụng cho các tác vụ phân loại. Đối với bài toán phân loại nhị phân, bạn chỉ cần một nơ-ron đầu ra duy nhất sử dụng hàm kích hoạt sigmoid: đầu ra sẽ là một số giữa 0 và 1, mà bạn có thể giải thích là xác suất ước tính của lớp dương. Xác suất ước tính của lớp âm bằng một trừ đi số đó.

MLP cũng có thể dễ dàng xử lý các tác vụ phân loại nhị phân đa nhãn (xem Chương 3). Ví dụ, bạn có thể có một hệ thống phân loại email dự đoán liệu mỗi email đến là thư hợp lệ hay thư rác, và đồng thời dự đoán liệu đó là email khẩn cấp hay không khẩn cấp. Trong trường hợp này, bạn sẽ cần hai nơ-ron đầu ra, cả hai đều sử dụng hàm kích hoạt sigmoid: nơ-ron đầu tiên sẽ xuất ra xác suất email là thư rác, và nơ-ron thứ hai sẽ xuất ra xác suất email là khẩn cấp. Tổng quát hơn, bạn sẽ dành một nơ-ron đầu ra cho mỗi lớp dương. Lưu ý rằng xác suất đầu ra không nhất thiết phải cộng lại bằng 1. Điều này cho phép mô hình xuất ra bất kỳ sự kết hợp nào của các nhãn: bạn có thể có thư hợp lệ không khẩn cấp, thư hợp lệ khẩn cấp, thư rác không khẩn cấp, và thậm chí có thể là thư rác khẩn cấp (mặc dù điều đó có lẽ sẽ là một lỗi).

Nếu mỗi trường hợp chỉ có thể thuộc về một lớp duy nhất, trong số ba hoặc nhiều lớp có thể (ví dụ: các lớp từ 0 đến 9 để phân loại hình ảnh chữ số), thì bạn cần có một nơ-ron đầu ra cho mỗi lớp, và bạn nên sử dụng hàm kích hoạt softmax cho toàn bộ lớp đầu ra (xem Hình 10-9). Hàm softmax (được giới thiệu trong Chương 4) sẽ đảm bảo rằng tất cả các xác suất ước tính nằm giữa 0 và 1 và chúng cộng lại bằng 1, vì các lớp là độc quyền. Như bạn đã thấy trong Chương 3, đây được gọi là phân loại đa lớp.

Đối với hàm mất mát, vì chúng ta đang dự đoán phân phối xác suất, mất mát entropy chéo (hoặc entropy X hoặc mất mát logarit, xem Chương 4) thường là một lựa chọn tốt.



Hình 10-9. Một MLP hiện đại (bao gồm ReLU và softmax) để phân loại

Scikit-Learn có một lớp `MLPClassifier` trong gói `sklearn.neural_network`. Nó gần như giống hệt với lớp `MLPRegressor`, ngoại trừ việc nó giảm thiểu entropy chéo thay vì MSE. Hãy thử nó ngay bây giờ, ví dụ trên tập dữ liệu iris. Đây gần như là một tác vụ tuyến tính, vì vậy một lớp duy nhất với 5 đến 10 nơ-ron sẽ đủ (đảm bảo chuẩn hóa các đặc trưng).

Bảng 10-2 tóm tắt kiến trúc điển hình của một MLP phân loại.

Bảng 10-2. Kiến trúc MLP phân loại điển hình

Siêu tham số	Phân loại nhị phân	Phân loại nhị phân đa nhãn	Phân loại đa lớp
# lớp ẩn	Thường từ 1 đến 5 lớp, tùy thuộc vào tác vụ		
# nơ-ron đầu ra	1	1 cho mỗi nhãn nhị phân	1 cho mỗi lớp
Kích hoạt lớp đầu ra	Sigmoid	Sigmoid	Softmax
Hàm mất mát	Entropy X	Entropy X	Entropy X

Bây giờ bạn đã có tất cả các khái niệm cần thiết để bắt đầu triển khai MLP với Keras!

Triển khai MLP với Keras

Keras là API học sâu cấp cao của TensorFlow: nó cho phép bạn xây dựng, huấn luyện, đánh giá và thực thi tất cả các loại mạng nơ-ron. Thư viện Keras ban đầu được François Chollet phát triển như một phần của dự án nghiên cứu và được phát hành dưới dạng một dự án mã

nguồn mở độc lập vào tháng 3 năm 2015. Nó nhanh chóng trở nên phổ biến nhờ tính dễ sử dụng, linh hoạt và thiết kế đẹp mắt.

Bây giờ hãy sử dụng Keras! Chúng ta sẽ bắt đầu bằng cách xây dựng một MLP để phân loại hình ảnh.

Xây dựng bộ phân loại hình ảnh bằng API tuần tự

Đầu tiên, chúng ta cần tải một tập dữ liệu. Chúng ta sẽ sử dụng Fashion MNIST, một tập dữ liệu thay thế trực tiếp cho MNIST (được giới thiệu trong Chương 3). Nó có định dạng chính xác giống như MNIST (70.000 hình ảnh thang độ xám 28×28 pixel mỗi hình, với 10 lớp), nhưng các hình ảnh này đại diện cho các mặt hàng thời trang chứ không phải chữ số viết tay, do đó mỗi lớp đa dạng hơn, và vấn đề này hóa ra thách thức hơn đáng kể so với MNIST. Ví dụ, một mô hình tuyến tính đơn giản đạt độ chính xác khoảng 92% trên MNIST, nhưng chỉ khoảng 83% trên Fashion MNIST.

Sử dụng Keras để tải tập dữ liệu

Keras cung cấp một số hàm tiện ích để lấy và tải các tập dữ liệu phổ biến, bao gồm MNIST, Fashion MNIST, và một vài tập dữ liệu khác. Hãy tải Fashion MNIST. Nó đã được xáo trộn và chia thành một tập huấn luyện (60.000 hình ảnh) và một tập kiểm tra (10.000 hình ảnh), nhưng chúng ta sẽ giữ lại 5.000 hình ảnh cuối cùng từ tập huấn luyện để xác thực:

```
import tensorflow as tf

fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
```

Khi tải MNIST hoặc Fashion MNIST bằng Keras thay vì Scikit-Learn, một điểm khác biệt quan trọng là mỗi hình ảnh được biểu diễn dưới dạng một mảng 28×28 thay vì một mảng 1D có kích thước 784. Hơn nữa, cường độ pixel được biểu diễn dưới dạng số nguyên (từ 0 đến 255) thay vì số thập phân (từ 0.0 đến 255.0). Hãy xem hình dạng và kiểu dữ liệu của tập huấn luyện:

```
>>> X_train.shape
(55000, 28, 28)
>>> X_train.dtype
dtype('uint8')
```

Để đơn giản, chúng ta sẽ chia cường độ pixel cho 255.0 để đưa chúng về phạm vi 0-1 (điều này cũng chuyển chúng thành số thập phân):

```
X_train, X_valid, X_test = X_train / 255., X_valid / 255., X_test / 255.
```

Với MNIST, khi nhân bằng 5, điều đó có nghĩa là hình ảnh đại diện cho chữ số viết tay 5. Dễ dàng. Tuy nhiên, đối với Fashion MNIST, chúng ta cần danh sách các tên lớp để biết chúng ta đang xử lý cái gì:

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

Ví dụ, hình ảnh đầu tiên trong tập huấn luyện đại diện cho một chiếc ủng cổ chân:

```
>>> class_names[y_train[0]]
'Ankle boot'
```

Hình 10-10 cho thấy một số mẫu từ tập dữ liệu Fashion MNIST.



Hình 10-10. Các mẫu từ Fashion MNIST

Tạo mô hình bằng API tuần tự

Bây giờ hãy xây dựng mạng nơ-ron! Đây là một MLP phân loại với hai lớp ẩn:

```
tf.random.set_seed(42)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(300, activation="relu"))
model.add(tf.keras.layers.Dense(100, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

Hãy đi qua đoạn mã này từng dòng:

- Đầu tiên, đặt hạt giống ngẫu nhiên của TensorFlow để làm cho kết quả có thể tái tạo: các trọng số ngẫu nhiên của các lớp ẩn và lớp đầu ra sẽ giống nhau mỗi khi bạn chạy notebook. Bạn cũng có thể chọn sử dụng hàm `tf.keras.utils.set_random_seed()`, chức năng này tiện lợi đặt các hạt giống ngẫu nhiên cho TensorFlow, Python (`random.seed()`), và NumPy (`np.random.seed()`).
- Dòng tiếp theo tạo một mô hình Sequential. Đây là loại mô hình Keras đơn giản nhất cho các mạng nơ-ron chỉ bao gồm một chồng các lớp được kết nối tuần tự. Đây được gọi là API tuần tự (sequential API).
- Tiếp theo, chúng ta xây dựng lớp đầu tiên (lớp Input) và thêm nó vào mô hình. Chúng ta chỉ định hình dạng đầu vào, không bao gồm kích thước batch, chỉ hình dạng của

các thể hiện. Keras cần biết hình dạng của các đầu vào để nó có thể xác định hình dạng của ma trận trọng số kết nối của lớp ẩn đầu tiên.

- Sau đó, chúng ta thêm một lớp Flatten. Vai trò của nó là chuyển đổi mỗi hình ảnh đầu vào thành một mảng 1D: ví dụ, nếu nó nhận được một batch có hình dạng [32, 28, 28], nó sẽ định hình lại thành [32, 784]. Nói cách khác, nếu nó nhận được dữ liệu đầu vào X, nó tính toán $X.reshape(-1, 784)$. Lớp này không có bất kỳ tham số nào; nó chỉ ở đó để thực hiện một số tiền xử lý đơn giản.
- Tiếp theo, chúng ta thêm một lớp ẩn Dense với 300 nơ-ron. Nó sẽ sử dụng hàm kích hoạt ReLU. Mỗi lớp Dense quản lý ma trận trọng số riêng của nó, chứa tất cả các trọng số kết nối giữa các nơ-ron và đầu vào của chúng. Nó cũng quản lý một vector các số hạng độ lệch (một cho mỗi nơ-ron). Khi nó nhận được một số dữ liệu đầu vào, nó tính toán theo Công thức 10-2.
- Sau đó, chúng ta thêm một lớp ẩn Dense thứ hai với 100 nơ-ron, cũng sử dụng hàm kích hoạt ReLU.
- Cuối cùng, chúng ta thêm một lớp đầu ra Dense với 10 nơ-ron (một cho mỗi lớp), sử dụng hàm kích hoạt softmax vì các lớp là độc quyền.

Thay vì thêm các lớp từng cái một như chúng ta vừa làm, thường thuận tiện hơn khi truyền một danh sách các lớp khi tạo mô hình Sequential. Bạn cũng có thể bỏ lớp Input và thay vào đó chỉ định `input_shape` trong lớp đầu tiên:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Phương thức `summary()` của mô hình hiển thị tất cả các lớp của mô hình, bao gồm tên của mỗi lớp (được tạo tự động trừ khi bạn đặt nó khi tạo lớp), hình dạng đầu ra của nó (None có nghĩa là kích thước batch có thể là bất kỳ thứ gì), và số lượng tham số của nó. Phần tóm tắt kết thúc với tổng số tham số, bao gồm các tham số có thể huấn luyện và không thể huấn luyện. Ở đây chúng ta chỉ có các tham số có thể huấn luyện (bạn sẽ thấy một số tham số không thể huấn luyện sau trong chương này):

```
>>> model.summary()
Model: "sequential"
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 300)	235500
dense_1 (Dense)	(None, 100)	30100
dense_2 (Dense)	(None, 10)	1010

Total params: 266,610
Trainable params: 266,610

```
Non-trainable params: 0
```

Lưu ý rằng các lớp Dense thường có rất nhiều tham số. Ví dụ, lớp ẩn đầu tiên có 784×300 trọng số kết nối, cộng với 300 số hạng độ lệch, tổng cộng là 235.500 tham số! Điều này mang lại cho mô hình khá nhiều sự linh hoạt để phù hợp với dữ liệu huấn luyện, nhưng nó cũng có nghĩa là mô hình có nguy cơ bị overfitting, đặc biệt là khi bạn không có nhiều dữ liệu huấn luyện. Chúng ta sẽ quay lại vấn đề này sau.

Mỗi lớp trong một mô hình phải có một tên duy nhất (ví dụ: “dense_2”). Bạn có thể đặt tên lớp một cách rõ ràng bằng cách sử dụng đối số name của hàm khởi tạo, nhưng nhìn chung, việc để Keras tự động đặt tên cho các lớp sẽ đơn giản hơn, như chúng ta vừa làm. Keras lấy tên lớp và chuyển đổi nó thành snake case (ví dụ: một lớp từ lớp MyCoolLayer được đặt tên là “my_cool_layer” theo mặc định). Keras cũng đảm bảo rằng tên là duy nhất trên toàn cầu, ngay cả giữa các mô hình, bằng cách thêm một chỉ số nếu cần, như trong “dense_2”. Nhưng tại sao nó lại bận tâm đến việc tạo các tên duy nhất giữa các mô hình? Chà, điều này giúp hợp nhất các mô hình dễ dàng mà không gặp xung đột tên.

Bạn có thể dễ dàng lấy danh sách các lớp của một mô hình bằng thuộc tính `layers`, hoặc sử dụng phương thức `get_layer()` để truy cập một lớp theo tên:

```
>>> model.layers
[<keras.layers.core.flatten.Flatten at 0x7fa1dea02250>,
<keras.layers.core.dense.Dense at 0x7fa1c8f42520>,
<keras.layers.core.dense.Dense at 0x7fa188be7ac0>,
<keras.layers.core.dense.dense.Dense at 0x7fa188be7fa0>]

>>> hidden1 = model.layers[1]

>>> hidden1.name
'dense'

>>> model.get_layer('dense') is hidden1
True
```

Tất cả các tham số của một lớp có thể được truy cập bằng các phương thức `get_weights()` và `set_weights()` của nó. Đối với một lớp Dense, điều này bao gồm cả trọng số kết nối và số hạng độ lệch:

```
>>> weights, biases = hidden1.get_weights()

>>> weights
array([[ 0.02448617, -0.00877795, -0.02189048, ...,  0.03859074,
        -0.06889391],
       [ 0.00476504, -0.03105379, -0.0586676 , ..., -0.02763776,
        -0.04165364],
       ...,
```

```
[ 0.07061854, -0.06960931, 0.07038955, ..., 0.00034875,
 0.02878492],
[-0.06022581, 0.01577859, -0.02585464, ..., 0.00272203,
-0.06793761]],
dtype=float32)

>>> weights.shape
(784, 300)

>>> biases
array([0., 0., 0., 0., 0., 0., 0., 0., 0., ..., 0., 0., 0.],
dtype=float32)

>>> biases.shape
(300,)
```

Lưu ý rằng lớp Dense đã khởi tạo trọng số kết nối một cách ngẫu nhiên (điều này cần thiết để phá vỡ tính đối xứng, như đã thảo luận trước đó), và các độ lệch được khởi tạo bằng 0, điều này là ổn. Nếu bạn muốn sử dụng một phương pháp khởi tạo khác, bạn có thể đặt `kernel_initializer` (kernel là một tên khác cho ma trận trọng số kết nối) hoặc `bias_initializer` khi tạo lớp. Chúng ta sẽ thảo luận chi tiết hơn về các trình khởi tạo trong Chương 11, và danh sách đầy đủ có tại <https://keras.io/api/layers/initializers>.

Biên dịch mô hình

Sau khi tạo một mô hình, bạn phải gọi phương thức `compile()` của nó để chỉ định hàm mất mát và bộ tối ưu hóa cần sử dụng. Tùy chọn, bạn có thể chỉ định một danh sách các số liệu bổ sung để tính toán trong quá trình huấn luyện và đánh giá:

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd",
metrics=["accuracy"])
```

Đoạn mã này cần được giải thích. Chúng ta sử dụng mất mát “`sparse_categorical_crossentropy`” vì chúng ta có các nhãn thừa thớt (tức là, đối với mỗi trường hợp, chỉ có một chỉ số lớp mục tiêu, từ 0 đến 9 trong trường hợp này), và các lớp là độc quyền. Nếu thay vào đó chúng ta có một xác suất mục tiêu cho mỗi lớp cho mỗi trường hợp (chẳng hạn như các vector one-hot, ví dụ: `[0., 0., 0., 1., 0., 0., 0., 0., 0., 0.]` để đại diện cho lớp 3), thì chúng ta sẽ cần sử dụng mất mát “`categorical_crossentropy`” thay thế. Nếu chúng ta đang thực hiện phân loại nhị phân hoặc phân loại nhị phân đa nhãn, thì chúng ta sẽ sử dụng hàm kích hoạt “`sigmoid`” trong lớp đầu ra thay vì hàm kích hoạt “`softmax`”, và chúng ta sẽ sử dụng mất mát “`binary_crossentropy`”.

Về bộ tối ưu hóa, “`sgd`” có nghĩa là chúng ta sẽ huấn luyện mô hình bằng cách sử dụng giảm độ dốc ngẫu nhiên (stochastic gradient descent). Nói cách khác, Keras sẽ thực hiện thuật toán backpropagation được mô tả trước đó (tức là, tự động đạo hàm ngược cộng với giảm độ dốc). Chúng ta sẽ thảo luận về các bộ tối ưu hóa hiệu quả hơn trong Chương 11. Chúng cải thiện giảm độ dốc, không phải tự động đạo hàm.

Cuối cùng, vì đây là một bộ phân loại, việc đo lường độ chính xác của nó trong quá trình huấn luyện và đánh giá là hữu ích, đó là lý do tại sao chúng ta đặt `metrics=["accuracy"]`.

Huấn luyện và đánh giá mô hình

Bây giờ mô hình đã sẵn sàng để được huấn luyện. Để làm điều này, chúng ta chỉ cần gọi phương thức `fit()` của nó:

```
>>> history = model.fit(X_train, y_train, epochs=30,
...                     validation_data=(X_valid, y_valid))
...
Epoch 1/30
1719/1719 [=====] - 2s 989us/step
- loss: 0.7220 - sparse_categorical_accuracy: 0.7649
- val_loss: 0.4959 - val_sparse_categorical_accuracy: 0.8332

Epoch 2/30
1719/1719 [=====] - 2s 964us/step
- loss: 0.4825 - sparse_categorical_accuracy: 0.8332
  - val_loss: 0.4567 - val_sparse_categorical_accuracy: 0.8384
[...]
Epoch 30/30
1719/1719 [=====] - 2s 963us/step
- loss: 0.2235 - sparse_categorical_accuracy: 0.9200
- val_loss: 0.3056 - val_sparse_categorical_accuracy: 0.8894
```

Chúng ta truyền cho nó các đặc trưng đầu vào (`X_train`) và các lớp mục tiêu (`y_train`), cũng như số epoch để huấn luyện (nếu không, nó sẽ mặc định chỉ là 1, điều này chắc chắn sẽ không đủ để hội tụ đến một giải pháp tốt). Chúng ta cũng truyền một tập xác thực (tùy chọn). Keras sẽ đo mất mát và các số liệu bổ sung trên tập này vào cuối mỗi epoch, điều này rất hữu ích để xem mô hình thực sự hoạt động tốt như thế nào. Nếu hiệu suất trên tập huấn luyện tốt hơn nhiều so với trên tập xác thực, mô hình của bạn có thể đang bị overfitting trên tập huấn luyện, hoặc có một lỗi, chẳng hạn như dữ liệu không khớp giữa tập huấn luyện và tập xác thực.

Và đó là tất cả! Mạng nơ-ron đã được huấn luyện. Ở mỗi epoch trong quá trình huấn luyện, Keras hiển thị số lượng mini-batch đã được xử lý ở phía bên trái của thanh tiến trình. Kích thước batch mặc định là 32, và vì tập huấn luyện có 55.000 hình ảnh, mô hình đi qua 1.719 batch mỗi epoch: 1.718 batch có kích thước 32 và 1 batch có kích thước 24. Sau thanh tiến trình, bạn có thể thấy thời gian huấn luyện trung bình cho mỗi mẫu, và mất mát và độ chính xác (hoặc bất kỳ số liệu bổ sung nào khác mà bạn yêu cầu) trên cả tập huấn luyện và tập xác thực. Lưu ý rằng mất mát huấn luyện đã giảm xuống, đây là một dấu hiệu tốt, và độ chính xác xác thực đạt 88,94% sau 30 epoch. Con số này hơi thấp hơn độ chính xác huấn luyện, vì vậy có một chút overfitting xảy ra, nhưng không đáng kể.

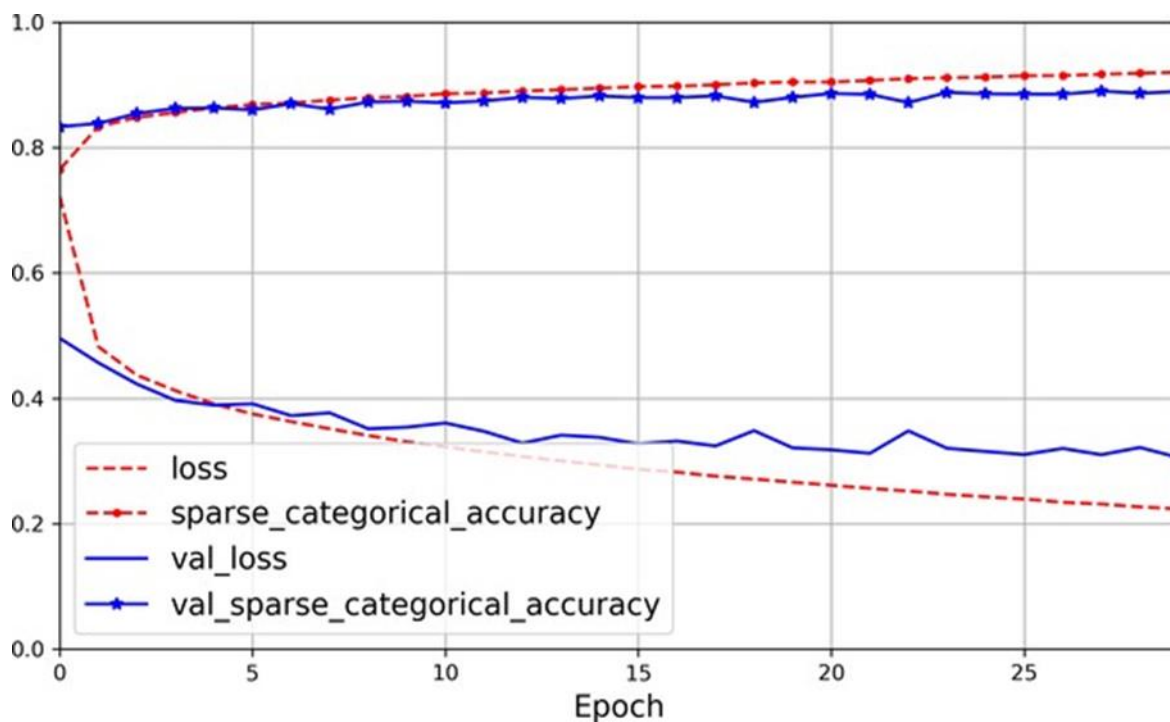
Nếu tập huấn luyện bị lệch nhiều, với một số lớp được biểu diễn quá mức và những lớp khác bị biểu diễn dưới mức, sẽ hữu ích nếu đặt đối số `class_weight` khi gọi phương thức `fit()`, để gán trọng số lớn hơn cho các lớp bị biểu diễn dưới mức và trọng số thấp hơn cho

các lớp được biểu diễn quá mức. Các trọng số này sẽ được Keras sử dụng khi tính toán mất mát. Nếu bạn cần trọng số cho mỗi trường hợp, hãy đặt đối số `sample_weight`. Nếu cả `class_weight` và `sample_weight` đều được cung cấp, thì Keras sẽ nhân chúng. Trọng số cho mỗi trường hợp có thể hữu ích, ví dụ, nếu một số trường hợp được dán nhãn bởi các chuyên gia trong khi những trường hợp khác được dán nhãn bằng nền tảng crowdsourcing: bạn có thể muốn gán trọng số lớn hơn cho những trường hợp trước. Bạn cũng có thể cung cấp trọng số mẫu (nhưng không phải trọng số lớp) cho tập xác thực bằng cách thêm chúng làm mục thứ ba trong tuple `validation_data`.

Phương thức `fit()` trả về một đối tượng `History` chứa các tham số huấn luyện (`history.params`), danh sách các epoch nó đã trải qua (`history.epoch`), và quan trọng nhất là một từ điển (`history.history`) chứa mất mát và các số liệu bổ sung mà nó đã đo được vào cuối mỗi epoch trên tập huấn luyện và trên tập xác thực (nếu có). Nếu bạn sử dụng từ điển này để tạo một `Pandas DataFrame` và gọi phương thức `plot()` của nó, bạn sẽ nhận được các đường cong học tập được hiển thị trong Hình 10-11:

```
import pandas as pd
import matplotlib.pyplot as plt

pd.DataFrame(history.history).plot(
    figsize=(8, 5), xlim=[0, 29], ylim=[0, 1], grid=True, xlabel="Epoch",
    style=["r--", "r--.", "b-", "b-*"])
plt.show()
```



Hình 10-11. Các đường cong học tập: mất mát huấn luyện trung bình và độ chính xác được đo qua mỗi epoch, và mất mát xác thực trung bình và độ chính xác được đo vào cuối mỗi epoch

Bạn có thể thấy rằng cả độ chính xác huấn luyện và độ chính xác xác thực đều tăng đều đặn trong quá trình huấn luyện, trong khi mất mát huấn luyện và mất mát xác thực giảm. Đây là một dấu hiệu tốt. Các đường cong xác thực ban đầu tương đối gần nhau, nhưng sau đó chúng ngày càng cách xa nhau, điều này cho thấy có một chút overfitting. Trong trường hợp cụ thể này, mô hình trông có vẻ hoạt động tốt hơn trên tập xác thực so với trên tập huấn luyện khi bắt đầu huấn luyện, nhưng điều đó thực sự không đúng. Lỗi xác thực được tính vào cuối mỗi epoch, trong khi lỗi huấn luyện được tính bằng cách sử dụng trung bình di chuyển trong mỗi epoch, vì vậy đường cong huấn luyện nên được dịch sang trái nửa epoch. Nếu bạn làm điều đó, bạn sẽ thấy rằng các đường cong huấn luyện và xác thực chồng lên nhau gần như hoàn hảo khi bắt đầu huấn luyện.

Hiệu suất của tập huấn luyện cuối cùng vượt qua hiệu suất xác thực, như thường lệ khi bạn huấn luyện đủ lâu. Bạn có thể thấy rằng mô hình chưa hội tụ hoàn toàn, vì mất mát xác thực vẫn đang giảm, vì vậy bạn có lẽ nên tiếp tục huấn luyện. Điều này đơn giản như việc gọi lại phương thức `fit()`, vì Keras chỉ tiếp tục huấn luyện từ nơi nó đã dừng lại: bạn sẽ có thể đạt được độ chính xác xác thực khoảng 89,8%, trong khi độ chính xác huấn luyện sẽ tiếp tục tăng lên đến 100% (điều này không phải lúc nào cũng xảy ra).

Nếu bạn không hài lòng với hiệu suất của mô hình, bạn nên quay lại và tinh chỉnh các siêu tham số. Điều đầu tiên cần kiểm tra là tốc độ học. Nếu điều đó không giúp ích, hãy thử một bộ tối ưu hóa khác (và luôn tinh chỉnh lại tốc độ học sau khi thay đổi bất kỳ siêu tham số nào). Nếu hiệu suất vẫn không tốt, thì hãy thử tinh chỉnh các siêu tham số mô hình như số lớp, số nơ-ron trên mỗi lớp và loại hàm kích hoạt để sử dụng cho mỗi lớp ẩn. Bạn cũng có thể thử tinh chỉnh các siêu tham số khác, chẳng hạn như kích thước batch (nó có thể được đặt trong phương thức `fit()` bằng cách sử dụng đối số `batch_size`, mặc định là 32). Chúng ta sẽ quay lại việc tinh chỉnh siêu tham số vào cuối chương này. Khi bạn hài lòng với độ chính xác xác thực của mô hình, bạn nên đánh giá nó trên tập kiểm tra để ước tính lỗi tổng quát hóa trước khi bạn triển khai mô hình vào sản xuất. Bạn có thể dễ dàng làm điều này bằng cách sử dụng phương thức `evaluate()` (nó cũng hỗ trợ một số đối số khác, chẳng hạn như `batch_size` và `sample_weight`; vui lòng kiểm tra tài liệu để biết thêm chi tiết):

```
>>> model.evaluate(X_test, y_test)
313/313 [=====] - 0s 626us/step
- loss: 0.3243 - sparse_categorical_accuracy: 0.8864
[0.32431697845458984, 0.8863999843597412]
```

Như bạn đã thấy trong Chương 2, việc đạt được hiệu suất hơi thấp hơn trên tập kiểm tra so với trên tập xác thực là điều phổ biến, bởi vì các siêu tham số được tinh chỉnh trên tập xác thực, không phải tập kiểm tra (tuy nhiên, trong ví dụ này, chúng ta không thực hiện bất kỳ tinh chỉnh siêu tham số nào, vì vậy độ chính xác thấp hơn chỉ là xui xẻo). Hãy nhớ chống lại sự cám dỗ điều chỉnh các siêu tham số trên tập kiểm tra, nếu không ước tính lỗi tổng quát hóa của bạn sẽ quá lạc quan.

Sử dụng mô hình để đưa ra dự đoán

Bây giờ hãy sử dụng phương thức `predict()` của mô hình để đưa ra dự đoán trên các thể hiện mới. Vì chúng ta không có các thể hiện mới thực tế, chúng ta sẽ chỉ sử dụng ba thể hiện đầu tiên của tập kiểm tra:

```
>>> X_new = X_test[:3]

>>> y_proba = model.predict(X_new)

>>> y_proba.round(2)
array([[0. , 0. , 0. , 0. , 0. , 0.01, 0. , 0.02, 0. , 0.97],
       [0. , 0. , 0.99, 0. , 0.01, 0. , 0. , 0. , 0. , 0. ],
       [0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ]],
      dtype=float32)
```

Đối với mỗi thể hiện, mô hình ước tính một xác suất cho mỗi lớp, từ lớp 0 đến lớp 9. Điều này tương tự như đầu ra của phương thức `predict_proba()` trong các bộ phân loại Scikit-Learn. Ví dụ, đối với hình ảnh đầu tiên, nó ước tính rằng xác suất của lớp 9 (ủng cổ chân) là 96%, xác suất của lớp 7 (giày thể thao) là 2%, xác suất của lớp 5 (dép) là 1%, và xác suất của các lớp khác là không đáng kể. Nói cách khác, nó rất tự tin rằng hình ảnh đầu tiên là giày dép, nhiều khả năng là ủng cổ chân nhưng cũng có thể là giày thể thao hoặc dép. Nếu bạn chỉ quan tâm đến lớp có xác suất ước tính cao nhất (ngay cả khi xác suất đó khá thấp), thì bạn có thể sử dụng phương thức `argmax()` để lấy chỉ số lớp có xác suất cao nhất cho mỗi thể hiện:

```
>>> import numpy as np

>>> y_pred = y_proba.argmax(axis=-1)

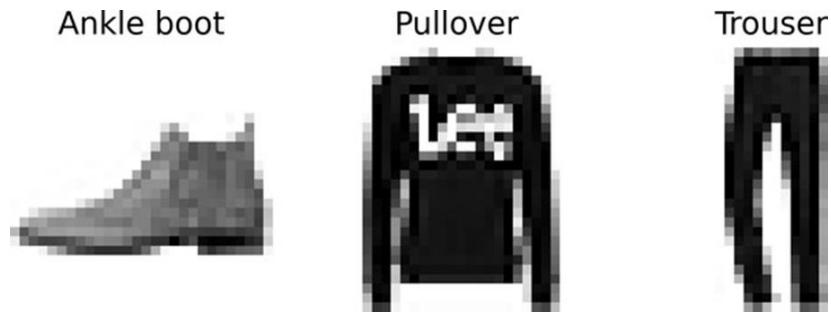
>>> y_pred
array([9, 2, 1])

>>> np.array(class_names)[y_pred]
array(['Ankle boot', 'Pullover', 'Trouser'], dtype='<U11')
```

Ở đây, bộ phân loại thực sự đã phân loại đúng cả ba hình ảnh (các hình ảnh này được hiển thị trong Hình 10-12):

```
>>> y_new = y_test[:3]

>>> y_new
array([9, 2, 1], dtype=uint8)
```



Hình 10-12. Các hình ảnh Fashion MNIST được phân loại đúng

Bây giờ bạn đã biết cách sử dụng API tuần tự để xây dựng, huấn luyện và đánh giá một MLP phân loại. Nhưng còn hồi quy thì sao?

Xây dựng một MLP hồi quy bằng API tuần tự

Hãy quay lại vấn đề nhà ở California và giải quyết nó bằng cách sử dụng cùng một MLP như trước, với 3 lớp ẩn gồm 50 nơ-ron mỗi lớp, nhưng lần này xây dựng nó bằng Keras.

Sử dụng API tuần tự để xây dựng, huấn luyện, đánh giá và sử dụng một MLP hồi quy khá giống với những gì chúng ta đã làm để phân loại. Sự khác biệt chính trong ví dụ mã sau đây là việc lớp đầu ra chỉ có một nơ-ron (vì chúng ta chỉ muốn dự đoán một giá trị duy nhất) và nó không sử dụng hàm kích hoạt, hàm mất mát là lỗi bình phương trung bình, số liệu là RMSE, và chúng ta đang sử dụng bộ tối ưu hóa Adam giống như MLPRegressor của Scikit-Learn đã làm. Hơn nữa, trong ví dụ này, chúng ta không cần một lớp Flatten, và thay vào đó chúng ta đang sử dụng một lớp Normalization làm lớp đầu tiên: nó thực hiện cùng một việc như StandardScaler của Scikit-Learn, nhưng nó phải được điều chỉnh cho dữ liệu huấn luyện bằng phương thức `adapt()` của nó trước khi bạn gọi phương thức `fit()` của mô hình. (Keras có các lớp tiền xử lý khác, sẽ được đề cập trong Chương 13).

Hãy cùng xem:

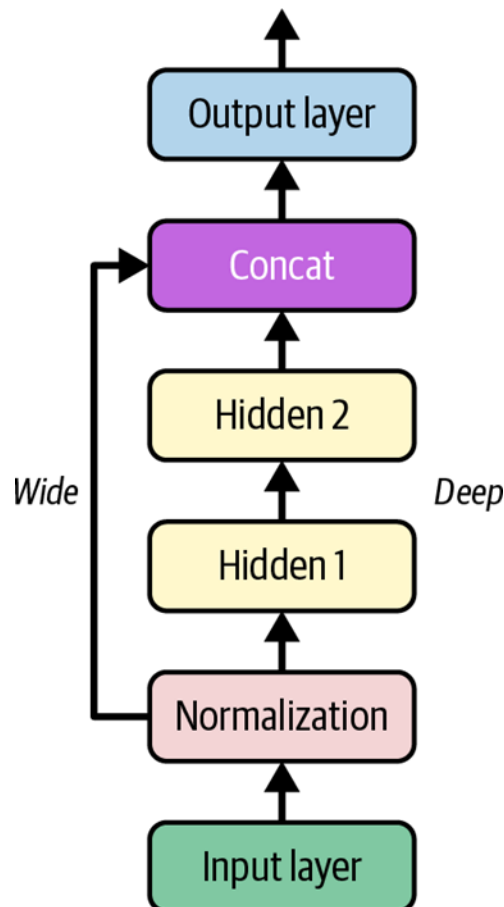
```
tf.random.set_seed(42)
norm_layer = tf.keras.layers.Normalization(input_shape=X_train.shape[1:])
model = tf.keras.Sequential([
    norm_layer,
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(1)
])
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer,
metrics=["RootMeanSquaredError"])
norm_layer.adapt(X_train)
history = model.fit(X_train, y_train, epochs=20,
validation_data=(X_valid, y_valid))
mse_test, rmse_test = model.evaluate(X_test, y_test)
```

```
X_new = X_test[:3]
y_pred = model.predict(X_new)
```

Như bạn có thể thấy, API tuần tự khá sạch sẽ và đơn giản. Tuy nhiên, mặc dù các mô hình Sequential cực kỳ phổ biến, đôi khi việc xây dựng mạng nơ-ron với cấu trúc phức tạp hơn, hoặc với nhiều đầu vào hoặc đầu ra, là hữu ích. Với mục đích này, Keras cung cấp API chức năng.

Xây dựng mô hình phức tạp bằng API chức năng

Một ví dụ về mạng nơ-ron không tuần tự là mạng nơ-ron Rộng & Sâu (Wide & Deep). Kiến trúc mạng nơ-ron này được giới thiệu trong một bài báo năm 2016 của Heng-Tze Cheng và cộng sự. Nó kết nối tất cả hoặc một phần các đầu vào trực tiếp với lớp đầu ra, như thể hiện trong Hình 10-13. Kiến trúc này cho phép mạng nơ-ron học cả các mẫu sâu (sử dụng đường dẫn sâu) và các quy tắc đơn giản (thông qua đường dẫn ngắn). Ngược lại, một MLP thông thường buộc tất cả dữ liệu phải chảy qua toàn bộ chồng lớp; do đó, các mẫu đơn giản trong dữ liệu có thể bị biến dạng bởi chuỗi các phép biến đổi này.



Hình 10-13. Mạng nơ-ron Rộng & Sâu

Hãy xây dựng một mạng nơ-ron như vậy để giải quyết bài toán nhà ở California:

```

normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])

```

Về cơ bản, năm dòng đầu tiên tạo tất cả các lớp chúng ta cần để xây dựng mô hình, sáu dòng tiếp theo sử dụng các lớp này giống như các hàm để đi từ đầu vào đến đầu ra, và dòng cuối cùng tạo một đối tượng Keras Model bằng cách trở đến đầu vào và đầu ra. Hãy xem chi tiết hơn đoạn mã này:

- Đầu tiên, chúng ta tạo năm lớp: một lớp Normalization để chuẩn hóa đầu vào, hai lớp Dense với 30 nơ-ron mỗi lớp, sử dụng hàm kích hoạt ReLU, một lớp Concatenate, và một lớp Dense nữa với một nơ-ron duy nhất cho lớp đầu ra, không có bất kỳ hàm kích hoạt nào.
- Tiếp theo, chúng ta tạo một đối tượng Input (tên biến input_ được sử dụng để tránh che khuất hàm input() tích hợp sẵn của Python). Đây là một đặc tả về loại đầu vào mà mô hình sẽ nhận được, bao gồm hình dạng của nó và tùy chọn kiểu dữ liệu của nó, mặc định là số thập phân 32 bit. Một mô hình thực sự có thể có nhiều đầu vào, như bạn sẽ thấy ngay sau đây.
- Sau đó, chúng ta sử dụng lớp Normalization giống như một hàm, truyền đối tượng Input cho nó. Đây là lý do tại sao nó được gọi là API chức năng. Lưu ý rằng chúng ta chỉ đang nói với Keras cách nó nên kết nối các lớp với nhau; chưa có dữ liệu thực tế nào được xử lý, vì đối tượng Input chỉ là một đặc tả dữ liệu. Nói cách khác, đó là một đầu vào mang tính biểu tượng. Đầu ra của lệnh gọi này cũng mang tính biểu tượng: normalized không lưu trữ bất kỳ dữ liệu thực tế nào, nó chỉ được sử dụng để xây dựng mô hình.
- Tương tự, chúng ta sau đó truyền normalized cho hidden_layer1, cái này xuất ra hidden1, và chúng ta truyền hidden1 cho hidden_layer2, cái này xuất ra hidden2.
- Cho đến nay, chúng ta đã kết nối các lớp một cách tuần tự, nhưng sau đó chúng ta sử dụng concat_layer để nối đầu vào và đầu ra của lớp ẩn thứ hai. Một lần nữa, chưa có dữ liệu thực tế nào được nối: tất cả đều mang tính biểu tượng, để xây dựng mô hình.
- Sau đó, chúng ta truyền concat cho output_layer, cái này cho chúng ta đầu ra cuối cùng.
- Cuối cùng, chúng ta tạo một Keras Model, chỉ định đầu vào và đầu ra cần sử dụng.

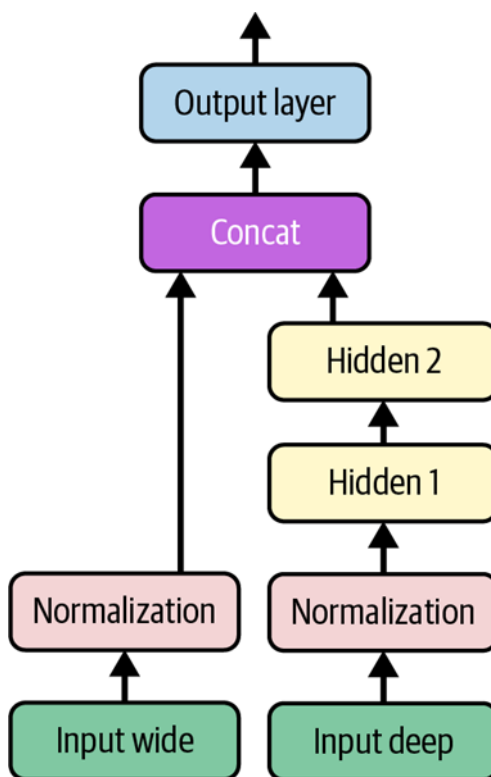
Khi bạn đã xây dựng mô hình Keras này, mọi thứ đều giống hệt như trước, vì vậy không cần lặp lại ở đây: bạn biên dịch mô hình, điều chỉnh lớp Normalization, huấn luyện mô hình, đánh giá nó và sử dụng nó để đưa ra dự đoán.

Nhưng điều gì sẽ xảy ra nếu bạn muốn gửi một tập hợp con các đặc trưng qua đường dẫn rộng và một tập hợp con khác (có thể chồng chéo) qua đường dẫn sâu, như minh họa trong Hình 10-14? Trong trường hợp này, một giải pháp là sử dụng nhiều đầu vào. Ví dụ, giả sử chúng ta muốn gửi năm đặc trưng qua đường dẫn rộng (đặc trưng 0 đến 4), và sáu đặc trưng qua đường dẫn sâu (đặc trưng 2 đến 7). Chúng ta có thể làm điều này như sau:

```
input_wide = tf.keras.layers.Input(shape=[5]) # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6]) # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()

norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])

output = tf.keras.layers.Dense(1)(concat)
model = tf.keras.Model(inputs=[input_wide, input_deep], outputs=[output])
```



Hình 10-14. Xử lý nhiều đầu vào

Có một vài điều cần lưu ý trong ví dụ này, so với ví dụ trước:

- Mỗi lớp Dense được tạo và gọi trên cùng một dòng. Đây là một thực hành phổ biến, vì nó làm cho mã ngắn gọn hơn mà không mất đi sự rõ ràng. Tuy nhiên, chúng ta không thể làm điều này với lớp Normalization vì chúng ta cần một tham chiếu đến lớp để có thể gọi phương thức `adapt()` của nó trước khi huấn luyện mô hình.
- Chúng ta đã sử dụng `tf.keras.layers.concatenate()`, cái này tạo một lớp Concatenate và gọi nó với các đầu vào đã cho.
- Chúng ta đã chỉ định `inputs=[input_wide, input_deep]` khi tạo mô hình, vì có hai đầu vào.

Bây giờ chúng ta có thể biên dịch mô hình như bình thường, nhưng khi chúng ta gọi phương thức `fit()`, thay vì truyền một ma trận đầu vào `X_train` duy nhất, chúng ta phải truyền một cặp ma trận (`X_train_wide`, `X_train_deep`), một cho mỗi đầu vào. Điều tương tự cũng đúng với `X_valid`, và cả `X_test` và `X_new` khi bạn gọi `evaluate()` hoặc `predict()`:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer,
metrics=["RootMeanSquaredError"])

X_train_wide, X_train_deep = X_train[:, :5], X_train[:, 2:]
X_valid_wide, X_valid_deep = X_valid[:, :5], X_valid[:, 2:]
X_test_wide, X_test_deep = X_test[:, :5], X_test[:, 2:]
X_new_wide, X_new_deep = X_test_wide[:3], X_test_deep[:3]

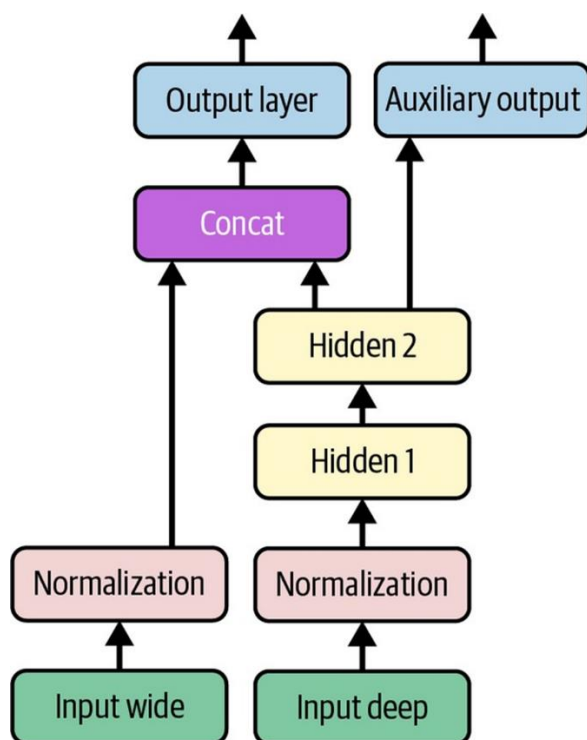
norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit(
    (X_train_wide, X_train_deep),
    y_train,
    epochs=20,
    validation_data=((X_valid_wide, X_valid_deep), y_valid)
)
mse_test = model.evaluate((X_test_wide, X_test_deep), y_test)
y_pred = model.predict((X_new_wide, X_new_deep))
```

Cũng có nhiều trường hợp sử dụng mà bạn có thể muốn có nhiều đầu ra:

- Tác vụ có thể yêu cầu điều đó. Ví dụ, bạn có thể muốn định vị và phân loại đối tượng chính trong một bức tranh. Đây vừa là tác vụ hồi quy vừa là tác vụ phân loại.
- Tương tự, bạn có thể có nhiều tác vụ độc lập dựa trên cùng một dữ liệu. Chắc chắn, bạn có thể huấn luyện một mạng nơ-ron cho mỗi tác vụ, nhưng trong nhiều trường hợp, bạn sẽ nhận được kết quả tốt hơn trên tất cả các tác vụ bằng cách huấn luyện một mạng nơ-ron duy nhất với một đầu ra cho mỗi tác vụ. Điều này là do mạng nơ-ron có thể học các đặc trưng trong dữ liệu hữu ích trên các tác vụ. Ví dụ, bạn có thể thực hiện phân loại đa tác vụ trên các bức ảnh khuôn mặt, sử dụng một đầu ra để

phân loại biểu cảm khuôn mặt của người đó (cười, ngạc nhiên, v.v.) và một đầu ra khác để xác định xem họ có đeo kính hay không.

- Một trường hợp sử dụng khác là một kỹ thuật chính quy hóa (nghĩa là, một ràng buộc huấn luyện với mục tiêu là giảm overfitting và do đó cải thiện khả năng tổng quát hóa của mô hình). Ví dụ, bạn có thể muốn thêm một đầu ra phụ trong kiến trúc mạng nơ-ron (xem Hình 10-15) để đảm bảo rằng phần cơ bản của mạng học được điều gì đó hữu ích một cách độc lập, mà không phụ thuộc vào phần còn lại của mạng.



Hình 10-15. Xử lý nhiều đầu ra, trong ví dụ này để thêm một đầu ra phụ để chính quy hóa

Thêm một đầu ra bổ sung khá dễ dàng: chúng ta chỉ cần kết nối nó với lớp thích hợp và thêm nó vào danh sách các đầu ra của mô hình. Ví dụ, đoạn mã sau xây dựng mạng được biểu diễn trong Hình 10-15:

```
# ... (Same as above, up to the main output layer)
output = tf.keras.layers.Dense(1)(concat)
aux_output = tf.keras.layers.Dense(1)(hidden2)

model = tf.keras.Model(inputs=[input_wide, input_deep],
                       outputs=[output, aux_output])
```

Mỗi đầu ra sẽ cần hàm mất mát riêng của nó. Do đó, khi chúng ta biên dịch mô hình, chúng ta nên truyền một danh sách các hàm mất mát. Nếu chúng ta truyền một hàm mất mát duy nhất, Keras sẽ giả định rằng cùng một hàm mất mát phải được sử dụng cho tất cả các đầu

ra. Theo mặc định, Keras sẽ tính toán tất cả các hàm mất mát và đơn giản là cộng chúng lại để có được tổng mất mát cuối cùng được sử dụng để huấn luyện. Vì chúng ta quan tâm nhiều hơn đến đầu ra chính hơn là đầu ra phụ (vì nó chỉ được sử dụng để chính quy hóa), chúng ta muốn gán trọng số lớn hơn nhiều cho mất mát của đầu ra chính. May mắn thay, có thể đặt tất cả các trọng số mất mát khi biên dịch mô hình:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss=("mse", "mse"), loss_weights=(0.9, 0.1),
              optimizer=optimizer,
              metrics=["RootMeanSquaredError"])
```

Bây giờ khi chúng ta huấn luyện mô hình, chúng ta cần cung cấp nhãn cho mỗi đầu ra. Trong ví dụ này, đầu ra chính và đầu ra phụ nên cố gắng dự đoán cùng một thứ, vì vậy chúng nên sử dụng cùng một nhãn. Vì vậy, thay vì truyền `y_train`, chúng ta cần truyền (`y_train`, `y_train`), hoặc một từ điển `{"output": y_train, "aux_output": y_train}` nếu các đầu ra được đặt tên là “output” và “aux_output”. Điều tương tự cũng áp dụng cho `y_valid` và `y_test`:

```
norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit(
    (X_train_wide, X_train_deep),
    (y_train, y_train),
    epochs=20,
    validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid))
)
```

Khi chúng ta đánh giá mô hình, Keras trả về tổng trọng số của các hàm mất mát, cũng như tất cả các hàm mất mát và số liệu riêng lẻ:

```
eval_results = model.evaluate((X_test_wide, X_test_deep), (y_test, y_test))
weighted_sum_of_losses, main_loss, aux_loss, main_rmse, aux_rmse =
eval_results
```

Tương tự, phương thức `predict()` sẽ trả về dự đoán cho mỗi đầu ra:

```
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

Phương thức `predict()` trả về một tuple, và nó không có đối số `return_dict` để thay vào đó lấy một từ điển. Tuy nhiên, bạn có thể tạo một từ điển bằng cách sử dụng `model.output_names`:

```
y_pred_tuple = model.predict((X_new_wide, X_new_deep))
y_pred = dict(zip(model.output_names, y_pred_tuple))
```

Như bạn có thể thấy, bạn có thể xây dựng tất cả các loại kiến trúc với API chức năng. Tiếp theo, chúng ta sẽ xem xét một cách cuối cùng để bạn có thể xây dựng các mô hình Keras.

Sử dụng Subclassing API để xây dựng mô hình động

Cả API tuần tự và API chức năng đều mang tính khai báo: bạn bắt đầu bằng cách khai báo các lớp bạn muốn sử dụng và cách chúng nên được kết nối, và sau đó bạn mới có thể bắt đầu cấp dữ liệu cho mô hình để huấn luyện hoặc suy luận. Điều này có nhiều lợi thế: mô hình có thể dễ dàng được lưu, sao chép và chia sẻ; cấu trúc của nó có thể được hiển thị và phân tích; framework có thể suy ra hình dạng và kiểm tra kiểu, do đó các lỗi có thể được phát hiện sớm (tức là trước khi bất kỳ dữ liệu nào đi qua mô hình). Việc gỡ lỗi cũng khá đơn giản, vì toàn bộ mô hình là một biểu đồ tính của các lớp. Nhưng mặt trái là: nó tĩnh. Một số mô hình liên quan đến các vòng lặp, hình dạng thay đổi, phân nhánh có điều kiện và các hành vi động khác. Đối với những trường hợp như vậy, hoặc đơn giản nếu bạn thích một phong cách lập trình mệnh lệnh hơn, API subclassing là dành cho bạn.

Với cách tiếp cận này, bạn kế thừa từ lớp `Model`, tạo các lớp bạn cần trong hàm tạo, và sử dụng chúng để thực hiện các phép tính bạn muốn trong phương thức `call()`. Ví dụ, tạo một thể hiện của lớp `WideAndDeepModel` sau đây cho chúng ta một mô hình tương đương với mô hình chúng ta vừa xây dựng bằng API chức năng:

```
class WideAndDeepModel(tf.keras.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs) # needed to support naming the model
        self.norm_layer_wide = tf.keras.layers.Normalization()
        self.norm_layer_deep = tf.keras.layers.Normalization()
        self.hidden1 = tf.keras.layers.Dense(units, activation=activation)
        self.hidden2 = tf.keras.layers.Dense(units, activation=activation)
        self.main_output = tf.keras.layers.Dense(1)
        self.aux_output = tf.keras.layers.Dense(1)

    def call(self, inputs):
        input_wide, input_deep = inputs
        norm_wide = self.norm_layer_wide(input_wide)
        norm_deep = self.norm_layer_deep(input_deep)
        hidden1 = self.hidden1(norm_deep)
        hidden2 = self.hidden2(hidden1)
        concat = tf.keras.layers.concatenate([norm_wide, hidden2])
        output = self.main_output(concat)
        aux_output = self.aux_output(hidden2)
        return output, aux_output

model = WideAndDeepModel(30, activation="relu", name="my_cool_model")
```

Ví dụ này trông giống như ví dụ trước, ngoại trừ việc chúng ta tách biệt việc tạo các lớp trong hàm tạo khỏi việc sử dụng chúng trong phương thức `call()`. Và chúng ta không cần tạo các đối tượng `Input`: chúng ta có thể sử dụng đối số `input` cho phương thức `call()`.

Bây giờ chúng ta đã có một thể hiện mô hình, chúng ta có thể biên dịch nó, điều chỉnh các lớp chuẩn hóa của nó (ví dụ: sử dụng `model.norm_layer_wide.adapt(...)`) và

`model.norm_layer_deep.adapt(...)`), huấn luyện nó, đánh giá nó và sử dụng nó để đưa ra dự đoán, chính xác như chúng ta đã làm với API chức năng.

Sự khác biệt lớn với API này là bạn có thể đưa vào phương thức `call()` bất cứ thứ gì bạn muốn: các vòng lặp `for`, các câu lệnh `if`, các phép toán TensorFlow cấp thấp – trí tưởng tượng của bạn là giới hạn (xem Chương 12)! Điều này làm cho nó trở thành một API tuyệt vời khi thử nghiệm các ý tưởng mới, đặc biệt đối với các nhà nghiên cứu. Tuy nhiên, sự linh hoạt bổ sung này đi kèm với một cái giá: kiến trúc mô hình của bạn bị ẩn trong phương thức `call()`, vì vậy Keras không thể dễ dàng kiểm tra nó; mô hình không thể được sao chép bằng `tf.keras.models.clone_model()`; và khi bạn gọi phương thức `summary()`, bạn chỉ nhận được danh sách các lớp, không có bất kỳ thông tin nào về cách chúng được kết nối với nhau. Hơn nữa, Keras không thể kiểm tra kiểu và hình dạng trước, và dễ mắc lỗi hơn. Vì vậy, trừ khi bạn thực sự cần sự linh hoạt bổ sung đó, bạn nên gắn bó với API tuần tự hoặc API chức năng.

Bây giờ bạn đã biết cách xây dựng và huấn luyện mạng nơ-ron bằng Keras, bạn sẽ muốn lưu chúng!

Lưu và khôi phục mô hình

Lưu một mô hình Keras đã được huấn luyện đơn giản nhất có thể:

```
model.save("my_keras_model", save_format="tf")
```

Khi bạn đặt `save_format="tf"`, Keras lưu mô hình bằng định dạng SavedModel của TensorFlow: đây là một thư mục (với tên đã cho) chứa nhiều tệp và thư mục con. Cụ thể, tệp `saved_model.pb` chứa kiến trúc và logic của mô hình dưới dạng biểu đồ tính toán được tuần tự hóa, vì vậy bạn không cần triển khai mã nguồn của mô hình để sử dụng nó trong sản xuất; SavedModel là đủ (bạn sẽ thấy cách hoạt động này trong Chương 12). Tệp `keras_metadata.pb` chứa thông tin bổ sung cần thiết của Keras. Thư mục con `variables` chứa tất cả các giá trị tham số (bao gồm trọng số kết nối, độ lệch, thống kê chuẩn hóa và tham số của bộ tối ưu hóa), có thể được chia thành nhiều tệp nếu mô hình rất lớn. Cuối cùng, thư mục `assets` có thể chứa các tệp bổ sung, chẳng hạn như mẫu dữ liệu, tên đặc trưng, tên lớp, v.v. Theo mặc định, thư mục `assets` trống. Vì bộ tối ưu hóa cũng được lưu, bao gồm các siêu tham số của nó và bất kỳ trạng thái nào nó có thể có, sau khi tải mô hình, bạn có thể tiếp tục huấn luyện nếu muốn.

Bạn thường sẽ có một tập lệnh huấn luyện mô hình và lưu nó, và một hoặc nhiều tập lệnh (hoặc dịch vụ web) tải mô hình và sử dụng nó để đánh giá hoặc để đưa ra dự đoán. Tải mô hình cũng dễ dàng như lưu nó:

```
model = tf.keras.models.load_model("my_keras_model")
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

Bạn cũng có thể sử dụng `save_weights()` và `load_weights()` để chỉ lưu và tải các giá trị tham số. Điều này bao gồm trọng số kết nối, độ lệch, thống kê tiền xử lý, trạng thái bộ tối ưu hóa, v.v. Các giá trị tham số được lưu trong một hoặc nhiều tệp như `my_weights.data-`

00004-of-00052, cộng với một tệp chỉ mục như `my_weights.index`. Lưu chỉ các trọng số nhanh hơn và sử dụng ít không gian đĩa hơn so với lưu toàn bộ mô hình, vì vậy nó hoàn hảo để lưu các điểm kiểm tra nhanh trong quá trình huấn luyện. Nếu bạn đang huấn luyện một mô hình lớn, và nó mất hàng giờ hoặc hàng ngày, thì bạn phải lưu các điểm kiểm tra thường xuyên trong trường hợp máy tính bị lỗi. Nhưng làm thế nào bạn có thể bảo phương thức `fit()` lưu các điểm kiểm tra? Sử dụng callback.

Sử dụng Callbacks

Phương thức `fit()` chấp nhận một đối số callbacks cho phép bạn chỉ định một danh sách các đối tượng mà Keras sẽ gọi trước và sau khi huấn luyện, trước và sau mỗi epoch, và thậm chí trước và sau khi xử lý mỗi batch. Ví dụ, callback `ModelCheckpoint` lưu các điểm kiểm tra của mô hình của bạn theo các khoảng thời gian đều đặn trong quá trình huấn luyện, theo mặc định vào cuối mỗi epoch:

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("my_checkpoints",
save_weights_only=True)
history = model.fit(..., callbacks=[checkpoint_cb])
```

Hơn nữa, nếu bạn sử dụng tập xác thực trong quá trình huấn luyện, bạn có thể đặt `save_best_only=True` khi tạo `ModelCheckpoint`. Trong trường hợp này, nó sẽ chỉ lưu mô hình của bạn khi hiệu suất của nó trên tập xác thực là tốt nhất cho đến nay. Bằng cách này, bạn không cần phải lo lắng về việc huấn luyện quá lâu và overfitting tập huấn luyện: chỉ cần khôi phục mô hình được lưu cuối cùng sau khi huấn luyện, và đây sẽ là mô hình tốt nhất trên tập xác thực. Đây là một cách để triển khai dừng sớm (được giới thiệu trong Chương 4), nhưng nó sẽ không thực sự dừng huấn luyện.

Một cách khác là sử dụng callback `EarlyStopping`. Nó sẽ làm gián đoạn quá trình huấn luyện khi nó không đo được tiến bộ nào trên tập xác thực trong một số epoch (được xác định bởi đối số `patience`), và nếu bạn đặt `restore_best_weights=True`, nó sẽ khôi phục lại mô hình tốt nhất vào cuối quá trình huấn luyện. Bạn có thể kết hợp cả hai callback để lưu các điểm kiểm tra của mô hình trong trường hợp máy tính của bạn bị lỗi, và làm gián đoạn quá trình huấn luyện sớm khi không còn tiến bộ nào nữa, để tránh lãng phí thời gian và tài nguyên và để giảm overfitting:

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10,
restore_best_weights=True)
history = model.fit(..., callbacks=[checkpoint_cb, early_stopping_cb])
```

Số lượng epoch có thể được đặt thành một giá trị lớn vì quá trình huấn luyện sẽ tự động dừng khi không còn tiến bộ (chỉ cần đảm bảo tốc độ học không quá nhỏ, nếu không nó có thể tiếp tục tiến bộ chậm cho đến cuối). Callback `EarlyStopping` sẽ lưu trữ trọng số của mô hình tốt nhất trong RAM, và nó sẽ khôi phục chúng cho bạn vào cuối quá trình huấn luyện.

Nếu bạn cần kiểm soát thêm, bạn có thể dễ dàng viết các callback tùy chỉnh của riêng mình. Ví dụ, callback tùy chỉnh sau đây sẽ hiển thị tỷ lệ giữa mất mát xác thực và mất mát huấn luyện trong quá trình huấn luyện (ví dụ: để phát hiện overfitting):

```
class PrintValTrainRatioCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs):
        ratio = logs["val_loss"] / logs["loss"]
        print(f"Epoch={epoch}, val/train={ratio:.2f}")
```

Như bạn có thể mong đợi, bạn có thể triển khai `on_train_begin()`, `on_train_end()`, `on_epoch_begin()`, `on_epoch_end()`, `on_batch_begin()`, và `on_batch_end()`. Các callback cũng có thể được sử dụng trong quá trình đánh giá và dự đoán, nếu bạn cần chúng (ví dụ: để gỡ lỗi). Để đánh giá, bạn nên triển khai `on_test_begin()`, `on_test_end()`, `on_test_batch_begin()`, hoặc `on_test_batch_end()`, được gọi bởi `evaluate()`. Để dự đoán, bạn nên triển khai `on_predict_begin()`, `on_predict_end()`, `on_predict_batch_begin()`, hoặc `on_predict_batch_end()`, được gọi bởi `predict()`.

Bây giờ chúng ta hãy xem một công cụ nữa mà bạn chắc chắn nên có trong bộ công cụ của mình khi sử dụng Keras: TensorBoard.

Sử dụng TensorBoard để trực quan hóa

TensorBoard là một công cụ trực quan hóa tương tác tuyệt vời mà bạn có thể sử dụng để xem các đường cong học tập trong quá trình huấn luyện, so sánh các đường cong và số liệu giữa nhiều lần chạy, trực quan hóa biểu đồ tính toán, phân tích thống kê huấn luyện, xem hình ảnh được tạo bởi mô hình của bạn, trực quan hóa dữ liệu đa chiều phức tạp được chiếu xuống 3D và tự động gom cụm cho bạn, phân tích hiệu suất mạng của bạn (tức là đo tốc độ để xác định các nút thắt cổ chai), và hơn thế nữa!

TensorBoard được cài đặt tự động khi bạn cài đặt TensorFlow. Tuy nhiên, bạn sẽ cần một plugin TensorBoard để trực quan hóa dữ liệu hồ sơ. Nếu bạn đã làm theo hướng dẫn cài đặt tại

<https://homl.info/install> để chạy mọi thứ cục bộ, thì bạn đã cài đặt plugin, nhưng nếu bạn đang sử dụng Colab, thì bạn phải chạy lệnh sau:

```
%pip install -q -U tensorboard-plugin-profile
```

Để sử dụng TensorBoard, bạn phải sửa đổi chương trình của mình để nó xuất dữ liệu bạn muốn trực quan hóa sang các tệp nhật ký nhị phân đặc biệt được gọi là tệp sự kiện. Mỗi bản ghi dữ liệu nhị phân được gọi là một bản tóm tắt. Máy chủ TensorBoard sẽ giám sát thư mục nhật ký, và nó sẽ tự động nhận các thay đổi và cập nhật các trực quan hóa: điều này cho phép bạn trực quan hóa dữ liệu trực tiếp (với một độ trễ ngắn), chẳng hạn như các đường cong học tập trong quá trình huấn luyện. Nói chung, bạn muốn trở máy chủ TensorBoard đến một thư mục nhật ký gốc và cấu hình chương trình của bạn để nó ghi vào một thư mục con khác mỗi khi nó chạy. Bằng cách này, cùng một thể hiện máy chủ TensorBoard sẽ cho phép bạn trực quan hóa và so sánh dữ liệu từ nhiều lần chạy chương trình của bạn, mà không làm mọi thứ bị lẫn lộn.

Hãy đặt tên thư mục nhật ký gốc là

my_logs, và hãy định nghĩa một hàm nhỏ tạo đường dẫn của thư mục con nhật ký dựa trên ngày và giờ hiện tại, để nó khác nhau ở mỗi lần chạy:

```
from pathlib import Path
from time import strftime

def get_run_logdir(root_logdir="my_logs"):
    return Path(root_logdir) / strftime("run_%Y_%m_%d_%H_%M_%S")

run_logdir = get_run_logdir() # e.g., my_logs/run_2022_08_01_17_25_59
``` [cite: 1]
```

Tin tốt là Keras cung cấp một callback `TensorBoard()` tiện lợi sẽ đảm nhiệm việc tạo thư mục nhật ký cho bạn (cùng với các thư mục cha của nó nếu cần), và nó sẽ tạo các tệp sự kiện và ghi các bản tóm tắt vào chúng trong quá trình huấn luyện. Nó sẽ đo mất mát và các số liệu huấn luyện và xác thực của mô hình của bạn (trong trường hợp này là MSE và RMSE), và nó cũng sẽ lập hồ sơ mạng nơ-ron của bạn. Nó rất dễ sử dụng: [cite: 1]

```
```python
tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir,
profile_batch=(100, 200))
history = model.fit([...], callbacks=[tensorboard_cb])
``` [cite: 1]
```

Đó là tất cả! Trong ví dụ này, nó sẽ lập hồ sơ mạng giữa các batch 100 và 200 trong epoch đầu tiên. Tại sao lại là 100 và 200? Chà, thường mất một vài batch để mạng nơ-ron "khởi động", vì vậy bạn không muốn lập hồ sơ quá sớm, và việc lập hồ sơ sử dụng tài nguyên, vì vậy tốt nhất là không nên làm điều đó cho mọi batch. [cite: 1]

Tiếp theo, hãy thử thay đổi tốc độ học từ 0.001 thành 0.002, và chạy lại mã, với một thư mục con nhật ký mới. Bạn sẽ có cấu trúc thư mục tương tự như sau: [cite: 1]

```
my_logs |— run_2022_08_01_17_25_59 | |— train | | |—
events.out.tfevents.1659331561.my_host_name.42042.0.v2 | | |—
events.out.tfevents.1659331562.my_host_name.profile-empty | | |— plugins | | |—
profile | | |— 2022_08_01_17_26_02 | | |— my_host_name.input_pipeline.pb | | |— [...]
| | |— validation | | |— events.out.tfevents.1659331562.my_host_name.42042.1.v2 | | |—
run_2022_08_01_17_31_12 | | |— [...]```
```

Có một thư mục cho mỗi lần chạy, mỗi thư mục chứa một thư mục con cho nhật ký huấn luyện và một cho nhật ký xác thực. Cả hai đều chứa các tệp sự kiện, và nhật ký huấn luyện cũng bao gồm các dấu vết lập hồ sơ.

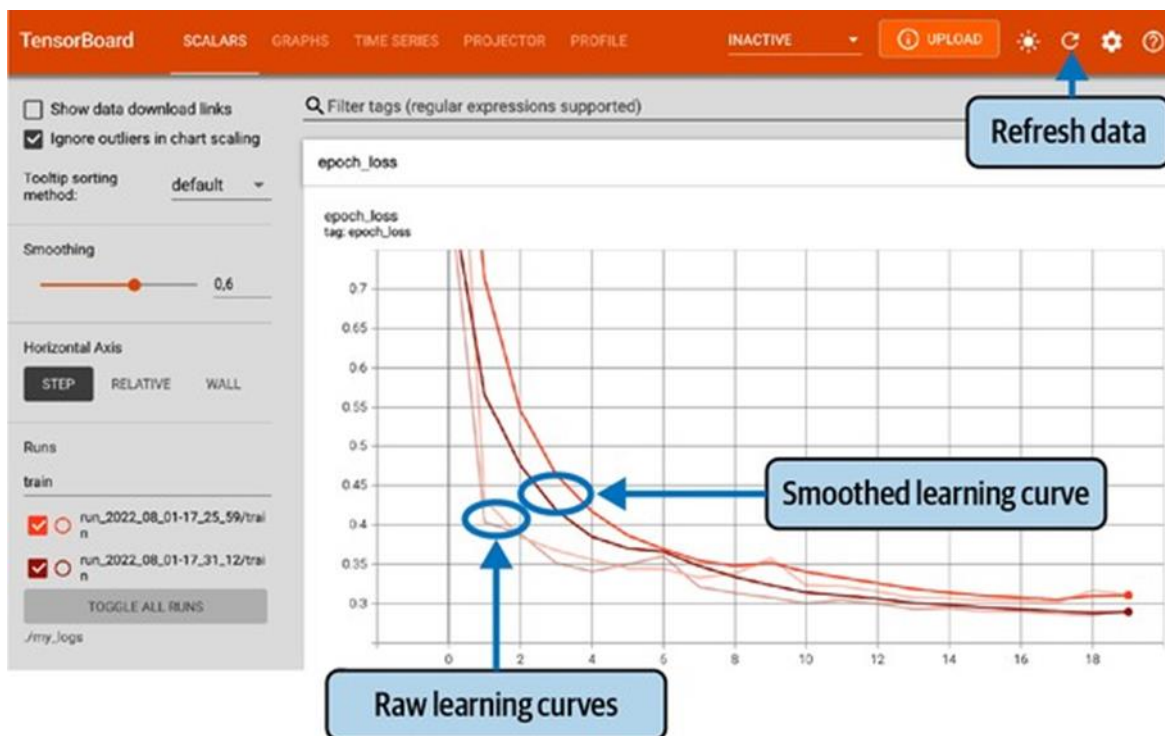
Bây giờ bạn đã có các tệp sự kiện sẵn sàng, đã đến lúc khởi động máy chủ TensorBoard. Điều này có thể được thực hiện trực tiếp trong Jupyter hoặc Colab bằng cách sử dụng tiện ích mở rộng Jupyter cho TensorBoard, được cài đặt cùng với thư viện TensorBoard. Tiện ích mở rộng này được cài đặt sẵn trong Colab. Đoạn mã sau tải tiện ích mở rộng Jupyter cho TensorBoard, và dòng thứ hai khởi động một máy chủ TensorBoard cho thư mục `my_logs`, kết nối với máy chủ này và hiển thị giao diện người dùng trực tiếp trong Jupyter. Máy chủ lắng nghe trên cổng TCP khả dụng đầu tiên lớn hơn hoặc bằng 6006 (hoặc bạn có thể đặt cổng bạn muốn bằng cách sử dụng tùy chọn

```
--port).
```

```
%load_ext tensorboard
```

```
%tensorboard --logdir=./my_logs
```

Bây giờ bạn sẽ thấy giao diện người dùng của TensorBoard. Nhấp vào tab SCALARS để xem các đường cong học tập (xem Hình 10-16). Ở phía dưới bên trái, chọn các nhật ký bạn muốn trực quan hóa (ví dụ: nhật ký huấn luyện từ lần chạy đầu tiên và thứ hai), và nhấp vào đại lượng vô hướng `epoch_loss`. Lưu ý rằng mất mát huấn luyện đã giảm đáng kể trong cả hai lần chạy, nhưng trong lần chạy thứ hai, nó giảm nhanh hơn một chút nhờ tốc độ học cao hơn.



Hình 10-16. Trực quan hóa các đường cong học tập với TensorBoard

Bạn cũng có thể trực quan hóa toàn bộ biểu đồ tính toán trong tab GRAPHS, trọng số đã học được chiếu xuống 3D trong tab PROJECTOR và các dấu vết lập hồ sơ trong tab PROFILE. Callback `TensorBoard()` có các tùy chọn để ghi thêm dữ liệu (xem tài liệu để biết



thêm chi tiết). Bạn có thể nhấp vào nút làm mới (🔄) ở phía trên bên phải để làm cho TensorBoard làm mới dữ liệu, và bạn có thể nhấp vào nút cài đặt (⚙️) để kích hoạt tự động làm mới và chỉ định khoảng thời gian làm mới.

Ngoài ra, TensorFlow cung cấp một API cấp thấp hơn trong gói `tf.summary`. Đoạn mã sau đây tạo một `SummaryWriter` bằng cách sử dụng hàm `create_file_writer()`, và nó sử dụng writer này làm ngữ cảnh Python để ghi nhật ký các đại lượng vô hướng, biểu đồ, hình ảnh, âm thanh và văn bản, tất cả đều có thể được trực quan hóa bằng TensorBoard:

```
test_logdir = get_run_logdir()

writer = tf.summary.create_file_writer(str(test_logdir))
with writer.as_default():
 for step in range(1, 1000 + 1):
 tf.summary.scalar("my_scalar", np.sin(step / 10), step=step)
 data = (np.random.randn(100) + 2) * step / 100 # gets larger
 tf.summary.histogram("my_hist", data, buckets=50, step=step)

 images = np.random.rand(2, 32, 32, 3) * step / 1000 # gets brighter
 tf.summary.image("my_images", images, step=step)

 texts = ["The step is " + str(step), "Its square is " + str(step **
2)]
 tf.summary.text("my_text", texts, step=step)

 sine_wave = tf.math.sin(tf.range(12000) / 48000 * 2 * np.pi * 1)
 audio = tf.reshape(tf.cast(sine_wave, tf.float32), [1, -1, 1])
 tf.summary.audio("my_audio", audio, sample_rate=48000, step=step)
 [cite: 1]
```

Nếu bạn chạy đoạn mã này và nhấp vào nút làm mới trong TensorBoard, bạn sẽ thấy một số tab xuất hiện: IMAGES, AUDIO, DISTRIBUTIONS, HISTOGRAMS và TEXT. Hãy thử nhấp vào tab IMAGES, và sử dụng thanh trượt phía trên mỗi hình ảnh để xem hình ảnh ở các bước thời gian khác nhau. Tương tự, hãy chuyển đến tab AUDIO và thử nghe âm thanh ở các bước thời gian khác nhau. Như bạn có thể thấy, TensorBoard là một công cụ hữu ích ngay cả ngoài TensorFlow hoặc học sâu. [cite: 1]

Tóm lại những gì bạn đã học được cho đến nay trong chương này: bạn đã biết mạng nơ-ron đến từ đâu, MLP là gì và cách bạn có thể sử dụng nó để phân loại và hồi quy, cách sử dụng API tuần tự của Keras để xây dựng MLP, và cách sử dụng API chức năng hoặc API subclassing để xây dựng các kiến trúc mô hình phức tạp hơn (bao gồm mô hình Rộng & Sâu, cũng như các mô hình có nhiều đầu vào và đầu ra). Bạn cũng đã học cách lưu và khôi phục mô hình và cách sử dụng các callback để lưu điểm kiểm tra, dừng sớm và hơn thế nữa. Cuối cùng, bạn đã học cách sử dụng TensorBoard để trực quan hóa. Bạn đã có thể tiếp tục và sử dụng mạng nơ-ron để giải quyết nhiều vấn đề! Tuy nhiên, bạn có thể tự hỏi làm

thế nào để chọn số lớp ẩn, số nơ-ron trong mạng và tất cả các siêu tham số khác. Hãy xem xét điều này bây giờ. [cite: 1]

## Tinh chỉnh các siêu tham số của mạng nơ-ron

Tính linh hoạt của mạng nơ-ron cũng là một trong những hạn chế chính của chúng: có rất nhiều siêu tham số cần điều chỉnh. Bạn không chỉ có thể sử dụng bất kỳ kiến trúc mạng nào có thể tưởng tượng được, mà ngay cả trong một MLP cơ bản, bạn có thể thay đổi số lớp, số nơ-ron và loại hàm kích hoạt sẽ sử dụng trong mỗi lớp, logic khởi tạo trọng số, loại bộ tối ưu hóa sẽ sử dụng, tốc độ học của nó, kích thước batch và hơn thế nữa. Làm thế nào để bạn biết sự kết hợp siêu tham số nào là tốt nhất cho tác vụ của bạn?

Một lựa chọn là chuyển đổi mô hình Keras của bạn thành một bộ ước lượng Scikit-Learn, và sau đó sử dụng GridSearchCV hoặc RandomizedSearchCV để tinh chỉnh các siêu tham số, như bạn đã làm trong Chương 2. Đối với điều này, bạn có thể sử dụng các lớp bao bọc KerasRegressor và KerasClassifier từ thư viện SciKeras (xem <https://github.com/adriangb/scikeras> để biết thêm chi tiết). Tuy nhiên, có một cách tốt hơn: bạn có thể sử dụng thư viện Keras Tuner, một thư viện tinh chỉnh siêu tham số cho các mô hình Keras. Nó cung cấp một số chiến lược tinh chỉnh, có khả năng tùy chỉnh cao và tích hợp tuyệt vời với TensorBoard. Hãy xem cách sử dụng nó.

Nếu bạn đã làm theo hướng dẫn cài đặt tại <https://homl.info/install> để chạy mọi thứ cục bộ, thì bạn đã cài đặt Keras Tuner, nhưng nếu bạn đang sử dụng Colab, bạn sẽ cần chạy `%pip install -q -U keras-tuner`. Tiếp theo, nhập `keras_tuner`, thường là dưới dạng `kt`, sau đó viết một hàm xây dựng, biên dịch và trả về một mô hình Keras. Hàm phải nhận một đối tượng `kt.HyperParameters` làm đối số, đối tượng này có thể được sử dụng để định nghĩa các siêu tham số (số nguyên, số thập phân, chuỗi, v.v.) cùng với phạm vi giá trị có thể có của chúng, và các siêu tham số này có thể được sử dụng để xây dựng và biên dịch mô hình. Ví dụ, hàm sau xây dựng và biên dịch một MLP để phân loại hình ảnh Fashion MNIST, sử dụng các siêu tham số như số lớp ẩn (`n_hidden`), số nơ-ron trên mỗi lớp (`n_neurons`), tốc độ học (`learning_rate`), và loại bộ tối ưu hóa sẽ sử dụng (`optimizer`):

```
import tensorflow as tf
import keras_tuner as kt

def build_model(hp):
 n_hidden = hp.Int("n_hidden", min_value=0, max_value=8, default=2)
 n_neurons = hp.Int("n_neurons", min_value=16, max_value=256)
 learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2,
sampling="log")
 optimizer = hp.Choice("optimizer", values=["sgd", "adam"])

 if optimizer == "sgd":
 optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)
 else:
 optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
```

```

model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten())
for _ in range(n_hidden):
 model.add(tf.keras.layers.Dense(n_neurons, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
model.compile(loss="sparse_categorical_crossentropy",
optimizer=optimizer, metrics=["accuracy"])
return model

```

Phần đầu tiên của hàm định nghĩa các siêu tham số. Ví dụ, `hp.Int("n_hidden", min_value=0, max_value=8, default=2)` kiểm tra xem một siêu tham số có tên “n\_hidden” đã có trong đối tượng `HyperParameters` `hp` hay chưa, và nếu có thì nó trả về giá trị của nó. Nếu không, thì nó đăng ký một siêu tham số số nguyên mới có tên “n\_hidden”, có các giá trị có thể có từ 0 đến 8 (bao gồm), và nó trả về giá trị mặc định, trong trường hợp này là 2 (khi `default` không được đặt, thì `min_value` được trả về). Siêu tham số “n\_neurons” được đăng ký theo cách tương tự. Siêu tham số “learning\_rate” được đăng ký dưới dạng số thập phân nằm trong khoảng từ  $10^{-4}$  đến  $10^{-2}$ , và vì `sampling="log"`, tốc độ học của tất cả các thang đo sẽ được lấy mẫu đều. Cuối cùng, siêu tham số `optimizer` được đăng ký với hai giá trị có thể có: “sgd” hoặc “adam” (giá trị mặc định là giá trị đầu tiên, trong trường hợp này là “sgd”). Tùy thuộc vào giá trị của `optimizer`, chúng ta tạo một bộ tối ưu hóa SGD hoặc một bộ tối ưu hóa Adam với tốc độ học đã cho.

Phần thứ hai của hàm chỉ xây dựng mô hình sử dụng các giá trị siêu tham số. Nó tạo một mô hình `Sequential` bắt đầu bằng một lớp `Flatten`, tiếp theo là số lượng lớp ẩn được yêu cầu (như được xác định bởi siêu tham số `n_hidden`) sử dụng hàm kích hoạt `ReLU`, và một lớp đầu ra với 10 nơ-ron (một cho mỗi lớp) sử dụng hàm kích hoạt `softmax`. Cuối cùng, hàm biên dịch mô hình và trả về nó.

Bây giờ nếu bạn muốn thực hiện tìm kiếm ngẫu nhiên cơ bản, bạn có thể tạo một tuner `kt.RandomSearch`, truyền hàm `build_model` vào hàm tạo, và gọi phương thức `search()` của tuner:

```

random_search_tuner = kt.RandomSearch(
 build_model, objective="val_accuracy", max_trials=5, overwrite=True,
 directory="my_fashion_mnist", project_name="my_rnd_search", seed=42)
random_search_tuner.search(X_train, y_train, epochs=10,
 validation_data=(X_valid, y_valid))

```

Tuner `RandomSearch` trước tiên gọi `build_model()` một lần với một đối tượng `Hyperparameters` trống, chỉ để thu thập tất cả các đặc tả siêu tham số. Sau đó, trong ví dụ này, nó chạy 5 lần thử; đối với mỗi lần thử, nó xây dựng một mô hình sử dụng các siêu tham số được lấy mẫu ngẫu nhiên trong phạm vi tương ứng của chúng, sau đó nó huấn luyện mô hình đó trong 10 epoch và lưu nó vào một thư mục con của thư mục `my_fashion_mnist/my_rnd_search`. Vì `overwrite=True`, thư mục `my_rnd_search` bị xóa trước khi quá trình huấn luyện bắt đầu. Nếu bạn chạy mã này lần thứ hai nhưng với `overwrite=False` và `max_trials=10`, tuner sẽ tiếp tục tinh chỉnh từ nơi nó đã dừng lại, chạy thêm 5 lần thử: điều này có nghĩa là bạn không phải chạy tất cả các lần thử trong một lần.

Cuối cùng, vì objective được đặt thành “val\_accuracy”, tuner ưu tiên các mô hình có độ chính xác thực cao hơn, vì vậy khi tuner đã hoàn thành tìm kiếm, bạn có thể nhận được các mô hình tốt nhất như sau:

```
top3_models = random_search_tuner.get_best_models(num_models=3)
best_model = top3_models[0]
```

Bạn cũng có thể gọi `get_best_hyperparameters()` để nhận `kt.HyperParameters` của các mô hình tốt nhất:

```
>>> top3_params = random_search_tuner.get_best_hyperparameters(num_trials=3)

>>> top3_params[0].values # best hyperparameter values
{'n_hidden': 5,
 'n_neurons': 70,
 'learning_rate': 0.00041268008323824807,
 'optimizer': 'adam'}
```

Mỗi tuner được hướng dẫn bởi một cái gọi là oracle: trước mỗi lần thử, tuner yêu cầu oracle cho biết lần thử tiếp theo nên là gì. Tuner `RandomSearch` sử dụng `RandomSearchOracle`, khá cơ bản: nó chỉ chọn lần thử tiếp theo một cách ngẫu nhiên, như chúng ta đã thấy trước đó. Vì oracle theo dõi tất cả các lần thử, bạn có thể yêu cầu nó cung cấp cho bạn cái tốt nhất, và bạn có thể hiển thị một bản tóm tắt của lần thử đó:

```
>>> best_trial = random_search_tuner.oracle.get_best_trials(num_trials=1)[0]

>>> best_trial.summary()
Trial summary Hyperparameters:
n_hidden: 5
n_neurons: 70
learning_rate: 0.00041268008323824807
optimizer: adam
Score: 0.8736000061035156
```

Điều này cho thấy các siêu tham số tốt nhất (như trước đó), cũng như độ chính xác thực. Bạn cũng có thể truy cập tất cả các số liệu trực tiếp:

```
>>> best_trial.metrics.get_last_value("val_accuracy")
0.8736000061035156
```

Nếu bạn hài lòng với hiệu suất của mô hình tốt nhất, bạn có thể tiếp tục huấn luyện nó trong vài epoch trên tập huấn luyện đầy đủ (`X_train_full` và `y_train_full`), sau đó đánh giá nó trên tập kiểm tra, và triển khai nó vào sản xuất (xem Chương 19):

```
best_model.fit(X_train_full, y_train_full, epochs=10)
test_loss, test_accuracy = best_model.evaluate(X_test, y_test)
```

Trong một số trường hợp, bạn có thể muốn tinh chỉnh các siêu tham số tiền xử lý dữ liệu, hoặc các đối số của `model.fit()`, chẳng hạn như kích thước batch. Đối với điều này, bạn phải sử dụng một kỹ thuật hơi khác: thay vì viết một hàm `build_model()`, bạn phải kế thừa

từ lớp `kt.HyperModel` và định nghĩa hai phương thức, `build()` và `fit()`. Phương thức `build()` thực hiện chính xác những gì hàm `build_model()` làm. Phương thức `fit()` nhận một đối tượng `HyperParameters` và một mô hình đã biên dịch làm đối số, cũng như tất cả các đối số của `model.fit()`, và huấn luyện mô hình và trả về đối tượng `History`. Quan trọng nhất, phương thức `fit()` có thể sử dụng các siêu tham số để quyết định cách tiền xử lý dữ liệu, điều chỉnh kích thước batch và hơn thế nữa. Ví dụ, lớp sau xây dựng cùng một mô hình như trước, với cùng các siêu tham số, nhưng nó cũng sử dụng một siêu tham số Boolean “normalize” để kiểm soát xem có chuẩn hóa dữ liệu huấn luyện trước khi huấn luyện mô hình hay không:

```
class MyClassificationHyperModel(kt.HyperModel):
 def build(self, hp):
 return build_model(hp)

 def fit(self, hp, model, X, y, **kwargs):
 if hp.Boolean("normalize"):
 norm_layer = tf.keras.layers.Normalization()
 X = norm_layer(X)
 return model.fit(X, y, **kwargs)
```

Sau đó, bạn có thể truyền một thể hiện của lớp này cho tuner bạn chọn, thay vì truyền hàm `build_model`. Ví dụ, hãy xây dựng một tuner `kt.Hyperband` dựa trên một thể hiện `MyClassificationHyperModel`:

```
hyperband_tuner = kt.Hyperband(
 MyClassificationHyperModel(), objective="val_accuracy", seed=42,
 max_epochs=10, factor=3, hyperband_iterations=2,
 overwrite=True, directory="my_fashion_mnist", project_name="hyperband")
```

Tuner này tương tự như lớp `HalvingRandomSearchCV` mà chúng ta đã thảo luận trong Chương 2: nó bắt đầu bằng cách huấn luyện nhiều mô hình khác nhau trong vài epoch, sau đó nó loại bỏ các mô hình tệ nhất và chỉ giữ lại các mô hình top  $1/\text{factor}$  (tức là, một phần ba trên cùng trong trường hợp này), lặp lại quá trình lựa chọn này cho đến khi chỉ còn một mô hình duy nhất. Đối số `max_epochs` kiểm soát số epoch tối đa mà mô hình tốt nhất sẽ được huấn luyện. Toàn bộ quá trình được lặp lại hai lần trong trường hợp này (`hyperband_iterations=2`). Tổng số epoch huấn luyện trên tất cả các mô hình cho mỗi lần lặp `hyperband` là khoảng  $\text{max\_epochs} * (\log(\text{max\_epochs}) / \log(\text{factor})) ** 2$ , vì vậy nó khoảng 44 epoch trong ví dụ này. Các đối số khác giống như đối với `kt.RandomSearch`.

Hãy chạy tuner `Hyperband` ngay bây giờ. Chúng ta sẽ sử dụng callback `TensorBoard`, lần này trỏ đến thư mục nhật ký gốc (tuner sẽ đảm nhiệm việc sử dụng một thư mục con khác cho mỗi lần thử), cũng như một callback `EarlyStopping`:

```
root_logdir = Path(hyperband_tuner.project_dir) / "tensorboard"
tensorboard_cb = tf.keras.callbacks.TensorBoard(root_logdir)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=2)
hyperband_tuner.search(X_train, y_train, epochs=10,
```

```
validation_data=(X_valid, y_valid),
callbacks=[early_stopping_cb, tensorboard_cb])
```

Bây giờ nếu bạn mở TensorBoard, trỏ `--logdir` đến thư mục `my_fashion_mnist/hyperband/tensorboard`, bạn sẽ thấy tất cả kết quả thử nghiệm khi chúng diễn ra. Đảm bảo truy cập tab HPARAMS: nó chứa bản tóm tắt tất cả các kết hợp siêu tham số đã được thử, cùng với các số liệu tương ứng. Lưu ý rằng có ba tab bên trong tab HPARAMS: chế độ xem bảng, chế độ xem tọa độ song song và chế độ xem ma trận biểu đồ phân tán. Ở phần dưới bên trái của bảng điều khiển, bỏ chọn tất cả các số liệu ngoại trừ `validation.epoch_accuracy`: điều này sẽ làm cho các biểu đồ rõ ràng hơn. Trong chế độ xem tọa độ song song, hãy thử chọn một phạm vi giá trị cao trong cột `validation.epoch_accuracy`: điều này sẽ lọc chỉ các kết hợp siêu tham số đạt hiệu suất tốt. Nhấp vào một trong các kết hợp siêu tham số, và các đường cong học tập tương ứng sẽ xuất hiện ở cuối trang. Dành một chút thời gian để xem xét từng tab; điều này sẽ giúp bạn hiểu tác động của từng siêu tham số đến hiệu suất, cũng như các tương tác giữa các siêu tham số.

Hyperband thông minh hơn tìm kiếm ngẫu nhiên thuần túy trong cách nó phân bổ tài nguyên, nhưng về cốt lõi, nó vẫn khám phá không gian siêu tham số một cách ngẫu nhiên; nó nhanh, nhưng thô. Tuy nhiên, Keras Tuner cũng bao gồm một tuner kt.BayesianOptimization: thuật toán này dần dần học được những vùng nào của không gian siêu tham số là hứa hẹn nhất bằng cách điều chỉnh một mô hình xác suất được gọi là quá trình Gaussian. Điều này cho phép nó dần dần phóng to vào các siêu tham số tốt nhất. Nhược điểm là thuật toán có các siêu tham số riêng: `alpha` đại diện cho mức độ nhiễu bạn mong đợi trong các phép đo hiệu suất trên các lần thử (mặc định là  $10^{-4}$ ), và `beta` chỉ định mức độ bạn muốn thuật toán khám phá, thay vì chỉ đơn giản là khai thác các vùng tốt đã biết của không gian siêu tham số (mặc định là 2.6). Ngoài ra, tuner này có thể được sử dụng giống như các tuner trước:

```
bayesian_opt_tuner = kt.BayesianOptimization(
 MyClassificationHyperModel(), objective="val_accuracy", seed=42,
 max_trials=10, alpha=1e-4, beta=2.6,
 overwrite=True, directory="my_fashion_mnist",
 project_name="bayesian_opt")
bayesian_opt_tuner.search(...)
```

Tinh chỉnh siêu tham số vẫn là một lĩnh vực nghiên cứu tích cực, và nhiều cách tiếp cận khác đang được khám phá. Ví dụ, hãy xem bài báo xuất sắc năm 2017 của DeepMind, nơi các tác giả đã sử dụng một thuật toán tiến hóa để cùng tối ưu hóa một tập hợp các mô hình và siêu tham số của chúng. Google cũng đã sử dụng một cách tiếp cận tiến hóa, không chỉ để tìm kiếm siêu tham số mà còn để khám phá tất cả các loại kiến trúc mô hình: nó hỗ trợ dịch vụ AutoML của họ trên Google Vertex AI (xem Chương 19). Thuật ngữ AutoML đề cập đến bất kỳ hệ thống nào đảm nhiệm một phần lớn quy trình làm việc ML.

Các thuật toán tiến hóa thậm chí đã được sử dụng thành công để huấn luyện các mạng nơ-ron riêng lẻ, thay thế giảm độ dốc phổ biến! Để biết ví dụ, xem bài đăng năm 2017 của Uber nơi các tác giả giới thiệu kỹ thuật Deep Neuroevolution của họ.

Nhưng bất chấp tất cả tiến bộ thú vị này và tất cả các công cụ và dịch vụ này, vẫn hữu ích khi có ý tưởng về những giá trị hợp lý cho mỗi siêu tham số để bạn có thể xây dựng một nguyên mẫu nhanh chóng và hạn chế không gian tìm kiếm. Các phần sau đây cung cấp hướng dẫn để chọn số lớp ẩn và nơ-ron trong một MLP và để chọn các giá trị tốt cho một số siêu tham số chính.

### *Số lượng lớp ẩn*

Đối với nhiều vấn đề, bạn có thể bắt đầu với một lớp ẩn duy nhất và nhận được kết quả hợp lý. Một MLP chỉ với một lớp ẩn về mặt lý thuyết có thể mô hình hóa ngay cả các hàm phức tạp nhất, miễn là nó có đủ nơ-ron. Nhưng đối với các vấn đề phức tạp, mạng sâu có hiệu quả tham số cao hơn nhiều so với mạng nông: chúng có thể mô hình hóa các hàm phức tạp bằng cách sử dụng số nơ-ron ít hơn theo cấp số mũ so với mạng nông, cho phép chúng đạt được hiệu suất tốt hơn nhiều với cùng một lượng dữ liệu huấn luyện.

Để hiểu tại sao, giả sử bạn được yêu cầu vẽ một khu rừng bằng một phần mềm vẽ nào đó, nhưng bạn bị cấm sao chép và dán bất cứ thứ gì. Sẽ mất một lượng thời gian khổng lồ: bạn sẽ phải vẽ từng cây một, từng nhánh một, từng lá một. Nếu bạn có thể vẽ một chiếc lá, sao chép và dán nó để vẽ một nhánh, sau đó sao chép và dán nhánh đó để tạo ra một cái cây, và cuối cùng sao chép và dán cái cây này để tạo ra một khu rừng, bạn sẽ hoàn thành ngay lập tức.

Dữ liệu thực tế thường được cấu trúc theo cách phân cấp như vậy, và mạng nơ-ron sâu tự động tận dụng thực tế này: các lớp ẩn thấp hơn mô hình hóa các cấu trúc cấp thấp (ví dụ: các đoạn thẳng có hình dạng và hướng khác nhau), các lớp ẩn trung gian kết hợp các cấu trúc cấp thấp này để mô hình hóa các cấu trúc cấp trung gian (ví dụ: hình vuông, hình tròn), và các lớp ẩn cao nhất và lớp đầu ra kết hợp các cấu trúc trung gian này để mô hình hóa các cấu trúc cấp cao (ví dụ: khuôn mặt).

Kiến trúc phân cấp này không chỉ giúp DNN hội tụ nhanh hơn đến một giải pháp tốt, mà nó còn cải thiện khả năng tổng quát hóa của chúng đối với các tập dữ liệu mới. Ví dụ, nếu bạn đã huấn luyện một mô hình để nhận dạng khuôn mặt trong ảnh và bây giờ bạn muốn huấn luyện một mạng nơ-ron mới để nhận dạng kiểu tóc, bạn có thể khởi động quá trình huấn luyện bằng cách tái sử dụng các lớp thấp hơn của mạng đầu tiên. Thay vì khởi tạo ngẫu nhiên các trọng số và độ lệch của một vài lớp đầu tiên của mạng nơ-ron mới, bạn có thể khởi tạo chúng bằng các giá trị của trọng số và độ lệch của các lớp thấp hơn của mạng đầu tiên. Bằng cách này, mạng sẽ không phải học từ đầu tất cả các cấu trúc cấp thấp xảy ra trong hầu hết các hình ảnh; nó sẽ chỉ phải học các cấu trúc cấp cao hơn (ví dụ: kiểu tóc). Đây được gọi là học chuyển giao.

Tóm lại, đối với nhiều vấn đề, bạn có thể bắt đầu chỉ với một hoặc hai lớp ẩn và mạng nơ-ron sẽ hoạt động tốt. Chẳng hạn, bạn có thể dễ dàng đạt độ chính xác trên 97% trên tập dữ

liệu MNIST chỉ với một lớp ẩn với vài trăm nơ-ron, và trên 98% độ chính xác với hai lớp ẩn với cùng tổng số nơ-ron, trong khoảng cùng một lượng thời gian huấn luyện. Đối với các vấn đề phức tạp hơn, bạn có thể tăng số lượng lớp ẩn cho đến khi bạn bắt đầu overfitting tập huấn luyện. Các tác vụ rất phức tạp, chẳng hạn như phân loại hình ảnh lớn hoặc nhận dạng giọng nói, thường yêu cầu mạng có hàng chục lớp (hoặc thậm chí hàng trăm, nhưng không phải các lớp kết nối đầy đủ, như bạn sẽ thấy trong Chương 14), và chúng cần một lượng lớn dữ liệu huấn luyện. Bạn sẽ hiếm khi phải huấn luyện các mạng như vậy từ đầu: việc tái sử dụng các phần của một mạng hiện đại đã được huấn luyện trước thực hiện một tác vụ tương tự phổ biến hơn nhiều. Quá trình huấn luyện sau đó sẽ nhanh hơn nhiều và yêu cầu ít dữ liệu hơn nhiều (chúng ta sẽ thảo luận về điều này trong Chương 11).

### *Số lượng nơ-ron trên mỗi lớp ẩn*

Số lượng nơ-ron trong các lớp đầu vào và đầu ra được xác định bởi loại đầu vào và đầu ra mà tác vụ của bạn yêu cầu. Ví dụ, tác vụ MNIST yêu cầu  $28 \times 28 = 784$  đầu vào và 10 nơ-ron đầu ra.

Đối với các lớp ẩn, trước đây thường có kích thước chúng để tạo thành một kim tự tháp, với số lượng nơ-ron ngày càng ít hơn ở mỗi lớp - lý do là nhiều đặc trưng cấp thấp có thể hợp nhất thành ít đặc trưng cấp cao hơn. Một mạng nơ-ron điển hình cho MNIST có thể có 3 lớp ẩn, lớp đầu tiên với 300 nơ-ron, lớp thứ hai với 200, và lớp thứ ba với 100. Tuy nhiên, thực hành này đã bị loại bỏ phần lớn vì dường như việc sử dụng cùng một số lượng nơ-ron trong tất cả các lớp ẩn hoạt động tốt trong hầu hết các trường hợp, hoặc thậm chí tốt hơn; hơn nữa, chỉ có một siêu tham số để điều chỉnh, thay vì một cho mỗi lớp. Điều đó nói lên rằng, tùy thuộc vào tập dữ liệu, đôi khi có thể hữu ích khi làm cho lớp ẩn đầu tiên lớn hơn các lớp khác.

Giống như số lượng lớp, bạn có thể thử tăng số lượng nơ-ron dần dần cho đến khi mạng bắt đầu overfitting. Ngoài ra, bạn có thể thử xây dựng một mô hình với số lượng lớp và nơ-ron nhiều hơn một chút so với thực tế bạn cần, sau đó sử dụng dừng sớm và các kỹ thuật chính quy hóa khác để ngăn nó overfitting quá nhiều.

Vincent Vanhoucke, một nhà khoa học tại Google, đã gọi đây là cách tiếp cận “quần co giãn”: thay vì lãng phí thời gian tìm kiếm chiếc quần vừa vặn hoàn hảo với kích thước của bạn, chỉ cần sử dụng quần co giãn lớn sẽ co lại đến kích thước phù hợp. Với cách tiếp cận này, bạn tránh được các lớp nút thắt cổ chai có thể làm hỏng mô hình của bạn. Thật vậy, nếu một lớp có quá ít nơ-ron, nó sẽ không có đủ sức mạnh biểu diễn để bảo toàn tất cả thông tin hữu ích từ các đầu vào (ví dụ: một lớp có hai nơ-ron chỉ có thể xuất ra dữ liệu 2D, vì vậy nếu nó nhận dữ liệu 3D làm đầu vào, một số thông tin sẽ bị mất). Bất kể phần còn lại của mạng lớn và mạnh đến đâu, thông tin đó sẽ không bao giờ được phục hồi.

### *Tốc độ học, kích thước Batch và các siêu tham số khác*

Số lượng lớp ẩn và nơ-ron không phải là các siêu tham số duy nhất bạn có thể điều chỉnh trong một MLP. Dưới đây là một số siêu tham số quan trọng nhất, cũng như các mẹo về cách đặt chúng:



**Tốc độ học** Tốc độ học được cho là siêu tham số quan trọng nhất. Nói chung, tốc độ học tối ưu bằng khoảng một nửa tốc độ học tối đa (tức là tốc độ học mà thuật toán huấn luyện phân kỳ, như chúng ta đã thấy trong Chương 4). Một cách để tìm tốc độ học tốt là huấn luyện mô hình trong vài trăm lần lặp, bắt đầu với tốc độ học rất thấp (ví dụ:  $10^{-5}$ ) và tăng dần nó lên một giá trị rất lớn (ví dụ: 10). Điều này được thực hiện bằng cách nhân tốc độ học với một hệ số không đổi ở mỗi lần lặp (ví dụ: bằng  $(10/10^{-5})^{1/500}$  để đi từ  $10^{-5}$  đến 10 trong 500 lần lặp). Nếu bạn vẽ mất mát theo hàm của tốc độ học (sử dụng thang logarit cho tốc độ học), bạn sẽ thấy nó giảm ban đầu. Nhưng sau một thời gian, tốc độ học sẽ quá lớn, do đó mất mát sẽ tăng trở lại: tốc độ học tối ưu sẽ thấp hơn một chút so với điểm mà mất mát bắt đầu tăng (thường thấp hơn khoảng 10 lần so với điểm ngoặt). Sau đó, bạn có thể khởi tạo lại mô hình của mình và huấn luyện nó bình thường bằng tốc độ học tốt này. Chúng ta sẽ xem xét nhiều kỹ thuật tối ưu hóa tốc độ học hơn trong Chương 11.

**Bộ tối ưu hóa** Chọn một bộ tối ưu hóa tốt hơn giảm độ dốc mini-batch đơn giản (và điều chỉnh các siêu tham số của nó) cũng khá quan trọng. Chúng ta sẽ xem xét một số bộ tối ưu hóa nâng cao trong Chương 11.

**Kích thước Batch** Kích thước batch có thể có tác động đáng kể đến hiệu suất và thời gian huấn luyện của mô hình của bạn. Lợi ích chính của việc sử dụng kích thước batch lớn là các bộ tăng tốc phần cứng như GPU có thể xử lý chúng hiệu quả, do đó thuật toán huấn luyện sẽ thấy nhiều thể hiện hơn mỗi giây. Do đó, nhiều nhà nghiên cứu và thực hành khuyên dùng kích thước batch lớn nhất có thể vừa với RAM GPU. Tuy nhiên, có một nhược điểm: trong thực tế, kích thước batch lớn thường dẫn đến mất ổn định huấn luyện, đặc biệt là khi bắt đầu huấn luyện, và mô hình kết quả có thể không tổng quát hóa tốt bằng mô hình được huấn luyện với kích thước batch nhỏ. Vào tháng 4 năm 2018, Yann LeCun thậm chí đã tweet “Bạn bè không để bạn bè sử dụng mini-batch lớn hơn 32”, trích dẫn một bài báo năm 2018 của Dominic Masters và Carlo Luschi kết luận rằng việc sử dụng các batch nhỏ (từ 2 đến 32) được ưu tiên hơn vì các batch nhỏ dẫn đến các mô hình tốt hơn trong thời gian huấn luyện ít hơn. Tuy nhiên, các nghiên cứu khác lại chỉ ra hướng ngược lại. Ví dụ, vào năm 2017, các bài báo của Elad Hoffer et al. và Priya Goyal et al. cho thấy có thể sử dụng kích thước batch rất lớn (lên đến 8.192) cùng với các kỹ thuật khác nhau như làm nóng tốc độ học (tức là bắt đầu huấn luyện với tốc độ học nhỏ, sau đó tăng dần lên, như đã thảo luận trong Chương 11) và để có được thời gian huấn luyện rất ngắn, mà không có bất kỳ khoảng cách tổng quát hóa nào. Vì vậy, một chiến lược là cố gắng sử dụng kích thước batch lớn, với làm nóng tốc độ học, và nếu quá trình huấn luyện không ổn định hoặc hiệu suất cuối cùng đáng thất vọng, thì hãy thử sử dụng kích thước batch nhỏ thay thế.

**Hàm kích hoạt** Chúng ta đã thảo luận cách chọn hàm kích hoạt ở đầu chương này: nói chung, hàm kích hoạt ReLU sẽ là một mặc định tốt cho tất cả các lớp ẩn, nhưng đối với lớp đầu ra thì nó thực sự phụ thuộc vào tác vụ của bạn.

**Số lần lặp** Trong hầu hết các trường hợp, số lần lặp huấn luyện thực sự không cần phải điều chỉnh: chỉ cần sử dụng dừng sớm thay thế.

Để biết thêm các thực hành tốt nhất liên quan đến việc điều chỉnh siêu tham số mạng nơ-ron, hãy xem bài báo xuất sắc năm 2018 của Leslie Smith.

Điều này kết thúc phần giới thiệu của chúng ta về mạng nơ-ron nhân tạo và việc triển khai chúng với Keras. Trong vài chương tiếp theo, chúng ta sẽ thảo luận về các kỹ thuật để huấn luyện các mạng rất sâu. Chúng ta cũng sẽ khám phá cách tùy chỉnh mô hình bằng API cấp thấp hơn của TensorFlow và cách tải và tiền xử lý dữ liệu hiệu quả bằng API `tf.data`. Và chúng ta sẽ đi sâu vào các kiến trúc mạng nơ-ron phổ biến khác: mạng nơ-ron tích chập để xử lý hình ảnh, mạng nơ-ron đệ quy và transformers cho dữ liệu tuần tự và văn bản, bộ tự mã hóa để học biểu diễn và mạng đối kháng tạo sinh để mô hình hóa và tạo dữ liệu.

## Bài tập

1. Sân chơi TensorFlow là một công cụ mô phỏng mạng nơ-ron tiện dụng được xây dựng bởi nhóm TensorFlow. Trong bài tập này, bạn sẽ huấn luyện một số bộ phân loại nhị phân chỉ bằng vài cú nhấp chuột, và điều chỉnh kiến trúc mô hình và các siêu tham số của nó để có được một số trực giác về cách mạng nơ-ron hoạt động và các siêu tham số của chúng làm gì. Dành một chút thời gian để khám phá những điều sau: a. Các mẫu được học bởi một mạng nơ-ron. Thử huấn luyện mạng nơ-ron mặc định bằng cách nhấp vào nút Chạy (trên cùng bên trái). Lưu ý cách nó nhanh chóng tìm thấy một giải pháp tốt cho tác vụ phân loại. Các nơ-ron trong lớp ẩn đầu tiên đã học các mẫu đơn giản, trong khi các nơ-ron trong lớp ẩn thứ hai đã học cách kết hợp các mẫu đơn giản của lớp ẩn đầu tiên thành các mẫu phức tạp hơn. Nói chung, càng có nhiều lớp, các mẫu càng có thể phức tạp hơn. b. Các hàm kích hoạt. Thử thay thế hàm kích hoạt tanh bằng hàm kích hoạt ReLU, và huấn luyện lại mạng. Lưu ý rằng nó tìm thấy một giải pháp thậm chí còn nhanh hơn, nhưng lần này các ranh giới là tuyến tính. Điều này là do hình dạng của hàm ReLU. c. Nguy cơ cực tiểu cục bộ. Sửa đổi kiến trúc mạng để chỉ có một lớp ẩn với ba nơ-ron. Huấn luyện nó nhiều lần (để đặt lại trọng số mạng, nhấp vào nút Đặt lại bên cạnh nút Phát). Lưu ý rằng thời gian huấn luyện thay đổi rất nhiều, và đôi khi nó thậm chí còn bị mắc kẹt trong một cực tiểu cục bộ. d. Điều gì xảy ra khi mạng nơ-ron quá nhỏ. Loại bỏ một nơ-ron để chỉ giữ lại hai nơ-ron. Lưu ý rằng mạng nơ-ron hiện không có khả năng tìm thấy một giải pháp tốt, ngay cả khi bạn thử nhiều lần. Mô hình có quá ít tham số và hệ thống hóa underfits tập huấn luyện. e. Điều gì xảy ra khi mạng nơ-ron đủ lớn. Đặt số lượng nơ-ron thành tám, và huấn luyện mạng nhiều lần. Lưu ý rằng bây giờ nó luôn nhanh chóng và không bao giờ bị mắc kẹt. Điều này làm nổi bật một phát hiện quan trọng trong lý thuyết mạng nơ-ron: các mạng nơ-ron lớn hiếm khi bị mắc kẹt trong các cực tiểu cục bộ, và ngay cả khi chúng bị mắc kẹt, các cực tiểu cục bộ này thường gần như tốt bằng cực đại toàn cục. Tuy nhiên, chúng vẫn có thể bị mắc kẹt trên các mặt phẳng dài trong một thời gian dài. f. Nguy cơ của các gradient biến mất trong mạng sâu. Chọn tập dữ liệu xoắn ốc (tập dữ liệu dưới cùng bên phải dưới "DATA"), và thay đổi kiến trúc mạng để có bốn lớp ẩn với tám nơ-ron mỗi lớp. Lưu ý rằng quá trình huấn luyện mất nhiều thời gian hơn và thường bị mắc kẹt trên các mặt phẳng trong thời gian dài. Cũng lưu ý rằng các nơ-ron trong các lớp cao nhất (ở bên phải) có xu hướng tiến hóa nhanh hơn các nơ-ron trong các lớp thấp nhất (ở bên trái). Vấn đề này, được gọi là vấn đề gradient biến mất, có thể được giảm nhẹ bằng cách khởi tạo trọng số tốt hơn và các kỹ thuật khác, các bộ tối ưu hóa tốt hơn (chẳng hạn như AdaGrad hoặc Adam), hoặc chuẩn hóa theo batch (được thảo luận trong

Chương 11). g. Đi xa hơn. Dành một giờ hoặc lâu hơn để chơi với các tham số khác và cảm nhận những gì chúng làm, để xây dựng một sự hiểu biết trực quan về mạng nơ-ron.

2. Vẽ một ANN sử dụng các nơ-ron nhân tạo ban đầu (giống như những nơ-ron trong Hình 10-3) tính toán  $A \oplus B$  (trong đó  $\oplus$  đại diện cho phép toán XOR). Gợi ý:  $A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$ .
3. Tại sao nói chung nên ưu tiên sử dụng bộ phân loại hồi quy logistic hơn là perceptron cổ điển (tức là một lớp duy nhất các đơn vị logic ngưỡng được huấn luyện bằng thuật toán huấn luyện perceptron)? Làm thế nào bạn có thể điều chỉnh một perceptron để làm cho nó tương đương với một bộ phân loại hồi quy logistic?
4. Tại sao hàm kích hoạt sigmoid là một thành phần quan trọng trong việc huấn luyện các MLP đầu tiên?
5. Kể tên ba hàm kích hoạt phổ biến. Bạn có thể vẽ chúng không?
6. Giả sử bạn có một MLP bao gồm một lớp đầu vào với 10 nơ-ron truyền qua, tiếp theo là một lớp ẩn với 50 nơ-ron nhân tạo, và cuối cùng là một lớp đầu ra với 3 nơ-ron nhân tạo. Tất cả các nơ-ron nhân tạo đều sử dụng hàm kích hoạt ReLU. a. Hình dạng của ma trận đầu vào  $X$  là gì? b. Hình dạng của ma trận trọng số  $W_h$  và vector độ lệch  $b_h$  của lớp ẩn là gì? c. Hình dạng của ma trận trọng số  $W_o$  và vector độ lệch  $b_o$  của lớp đầu ra là gì? d. Hình dạng của ma trận đầu ra  $Y$  của mạng là gì? e. Viết phương trình tính ma trận đầu ra  $Y$  của mạng dưới dạng hàm của  $X$ ,  $W_h$ ,  $b_h$ ,  $W_o$ , và  $b_o$ .
7. Bạn cần bao nhiêu nơ-ron trong lớp đầu ra nếu bạn muốn phân loại email thành thư rác hoặc thư hợp lệ? Bạn nên sử dụng hàm kích hoạt nào trong lớp đầu ra? Nếu thay vào đó bạn muốn giải quyết MNIST, bạn cần bao nhiêu nơ-ron trong lớp đầu ra, và bạn nên sử dụng hàm kích hoạt nào? Còn đối với việc để mạng của bạn dự đoán giá nhà, như trong Chương 2 thì sao?
8. Backpropagation là gì và nó hoạt động như thế nào? Sự khác biệt giữa backpropagation và tự động đạo hàm ngược là gì?
9. Bạn có thể liệt kê tất cả các siêu tham số bạn có thể điều chỉnh trong một MLP cơ bản không? Nếu MLP overfitting dữ liệu huấn luyện, làm thế nào bạn có thể điều chỉnh các siêu tham số này để cố gắng giải quyết vấn đề?
10. Huấn luyện một MLP sâu trên tập dữ liệu MNIST (bạn có thể tải nó bằng cách sử dụng `tf.keras.datasets.mnist.load_data()`). Xem liệu bạn có thể đạt được độ chính xác trên 98% bằng cách điều chỉnh thủ công các siêu tham số. Thử tìm kiếm tốc độ học tối ưu bằng cách sử dụng cách tiếp cận được trình bày trong chương này (tức là, bằng cách tăng tốc độ học theo cấp số mũ, vẽ biểu đồ mất mát và tìm điểm mà mất mát tăng vọt). Tiếp theo, thử điều chỉnh các siêu tham số bằng Keras Tuner với tất cả các tính năng - lưu điểm kiểm tra, sử dụng dừng sớm và vẽ các đường cong học tập bằng TensorBoard.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.